

SAVEETHA SCHOOL OF ENGINEERING

**DEPARTMENT OF ARTIFICIAL INTELLIGENCE
AND DATA SCIENCE**

NAME : HASWANTH B

REG NO : 192424091

CSA1411 - COMPILER DESIGN

LABORATORY MANUAL

TABLE OF CONTENTS

1. Lexical Analyzer – identifiers, constants and operators
2. Lexical Analyzer – single-line and multi-line comments
3. Lexical Analyzer – ignore spaces, tabs, new lines and comments
4. Lexical Analyzer – validate arithmetic operators
5. Lexical Analyzer – count whitespaces and newlines
6. Lexical Analyzer – validate identifiers
7. FIRST() – predictive parser
8. FOLLOW() – predictive parser
9. Eliminate left recursion from a CFG
10. Eliminate left factoring from a CFG
11. Symbol table operations
12. Recursive descent parsing
13. Top-down parsing to validate a grammar
14. Three-address code generation
15. Lexical Analyzer – count characters, words and lines in a file
16. Back end of the compiler
17. LEADING() – operator precedence parser
18. TRAILING() – operator precedence parser
19. LEX – count characters, lines and words in a C file
20. LEX – print constants in a C source file
21. Algorithm – count vowels in a sentence
22. LEX – print HTML tags
23. LEX – add line numbers to a C program
24. LEX – count and remove comments from a C program

25. LEX – identify capital words
26. LEX – validate an email address
27. LEX – convert abc to ABC
28. LEX – validate mobile numbers
29. LEX/FLEX – separate C tokens with captions
30. Algorithm – count consonants in a sentence
31. LEX – separate keywords and identifiers
32. LEX – count positive and negative numbers
33. LEX – validate a URL
34. LEX – validate date of birth
35. LEX – check whether input is a digit
36. LEX – basic mathematical operations
37. LEX – frequency of a word in a sentence
38. LEX – length of the longest word
39. LEX – replace one word with another
40. FLEX – token separation for a C program

EXPERIMENT 1: Lexical Analyzer – identifiers, constants and operators

AIM

A lexical analyzer that ignores redundant spaces, tabs, new lines and comments, and identifies identifiers, constants and operators in a C program.

ALGORITHM

- Read the source program character by character.
- Skip spaces, tabs, new lines and comments.
- If a letter or underscore is found, collect the complete identifier.
- If a digit is found, collect the complete numeric constant.
- If an operator character is found, classify it as an operator.
- Print each recognized token with its category.

PROGRAM

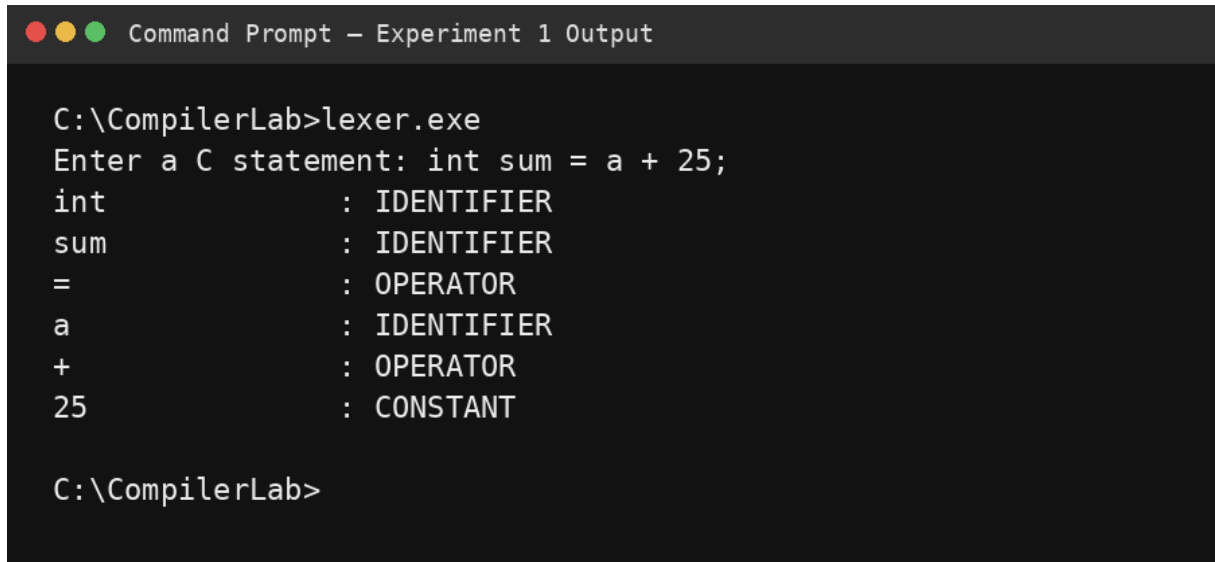
```
#include <stdio.h>
#include <ctype.h>
#include <string.h>

int main(void) {
    char s[1000];
    int i = 0;
    printf("Enter a C statement: ");
    fgets(s, sizeof(s), stdin);

    while (s[i] != '\0') {
        if (isspace((unsigned char)s[i])) { i++; continue; }

        if (isalpha((unsigned char)s[i]) || s[i] == '_' ) {
            char tok[100];
            int j = 0;
            while (isalnum((unsigned char)s[i]) || s[i] == '_')
                tok[j++] = s[i++];
            tok[j] = '\0';
            printf("%-15s : IDENTIFIER\n", tok);
        } else if (isdigit((unsigned char)s[i])) {
            char tok[100];
            int j = 0;
            while (isdigit((unsigned char)s[i]) || s[i] == '.')
                tok[j++] = s[i++];
            tok[j] = '\0';
            printf("%-15s : CONSTANT\n", tok);
        } else if (strchr("+-*/%=<>!", s[i])) {
            printf("%-15c : OPERATOR\n", s[i++]);
        } else {
            i++;
        }
    }
    return 0;
}
```

OUTPUT



```
C:\CompilerLab>lexer.exe
Enter a C statement: int sum = a + 25;
int           : IDENTIFIER
sum           : IDENTIFIER
=             : OPERATOR
a             : IDENTIFIER
+             : OPERATOR
25            : CONSTANT

C:\CompilerLab>
```

RESULT

Thus, the lexical analyzer successfully identified identifiers, constants and operators while ignoring redundant white space.

EXPERIMENT 2: Lexical Analyzer – single-line and multi-line comments

AIM

To determine whether a given C source line is a single-line comment, a multi-line comment, or ordinary code.

ALGORITHM

- Remove leading white space conceptually for classification.
- If the line starts with //, classify it as a single-line comment.
- If the line starts with /*, search for */ and classify it as a multi-line comment when present.
- Otherwise classify it as a non-comment line.
- Display the classification.

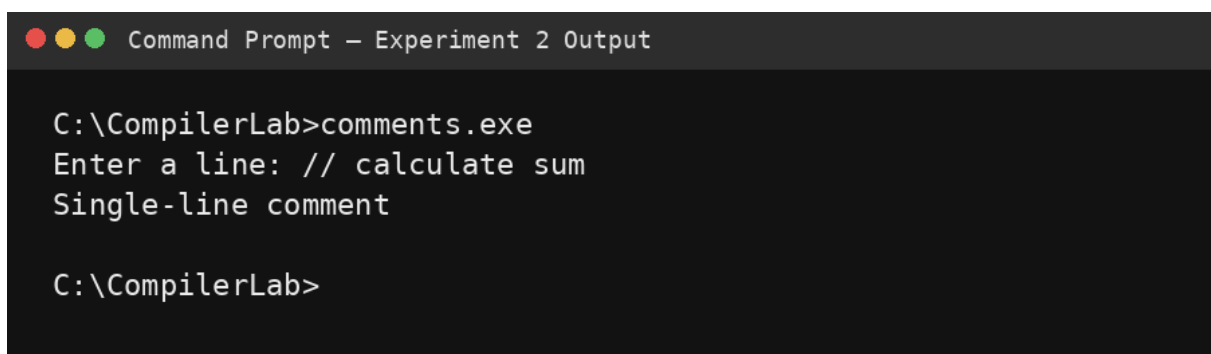
PROGRAM

```
#include <stdio.h>
#include <string.h>

int main(void) {
    char s[500];
    printf("Enter a line: ");
    fgets(s, sizeof(s), stdin);

    if (strncmp(s, "//", 2) == 0)
        printf("Single-line comment\n");
    else if (strncmp(s, "/*", 2) == 0) {
        if (strstr(s + 2, "*/"))
            printf("Multi-line comment\n");
        else
            printf("Start of a multi-line comment\n");
    } else
        printf("Not a comment\n");
    return 0;
}
```

OUTPUT



```
Command Prompt - Experiment 2 Output

C:\CompilerLab>comments.exe
Enter a line: // calculate sum
Single-line comment

C:\CompilerLab>
```

RESULT

Thus, the program correctly identifies the specified C comment forms.

EXPERIMENT 3: Lexical Analyzer – ignore spaces, tabs, new lines and comments

AIM

To design a lexical analyzer that removes redundant white space and comments before processing the remaining source characters.

ALGORITHM

- Read the complete input text.
- Skip white-space characters.
- When // is encountered, skip until the newline.
- When /* is encountered, skip until */.
- Print all remaining characters in their original order.
- Verify that comments and redundant white space are absent.

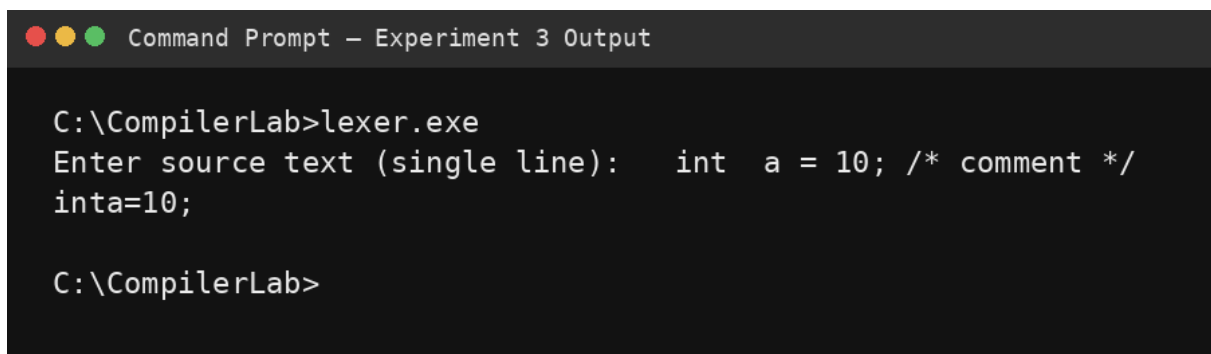
PROGRAM

```
#include <stdio.h>

int main(void) {
    char s[1000];
    int i = 0, in_block = 0;
    printf("Enter source text (single line): ");
    fgets(s, sizeof(s), stdin);

    while (s[i]) {
        if (!in_block && s[i]=='/' && s[i+1]=='/') break;
        if (!in_block && s[i]=='/' && s[i+1]=='*') {
            in_block = 1; i += 2; continue;
        }
        if (in_block && s[i]=='*' && s[i+1]=='/') {
            in_block = 0; i += 2; continue;
        }
        if (!in_block && s[i]!='\n' && s[i]!='\t' && s[i]!=' ')
            putchar(s[i]);
        i++;
    }
    putchar('\n');
    return 0;
}
```

OUTPUT



```
Command Prompt - Experiment 3 Output

C:\CompilerLab>lexer.exe
Enter source text (single line):  int  a = 10; /* comment */
inta=10;

C:\CompilerLab>
```

RESULT

Thus, redundant spaces and comments were ignored successfully.

EXPERIMENT 4: Lexical Analyzer – validate arithmetic operators

AIM

To recognize and validate the arithmetic operators +, -, * and /.

ALGORITHM

- Read an input character.
- Compare it with +, -, * and /.
- If a match is found, print the corresponding arithmetic operator.
- Otherwise report that it is not one of the required operators.

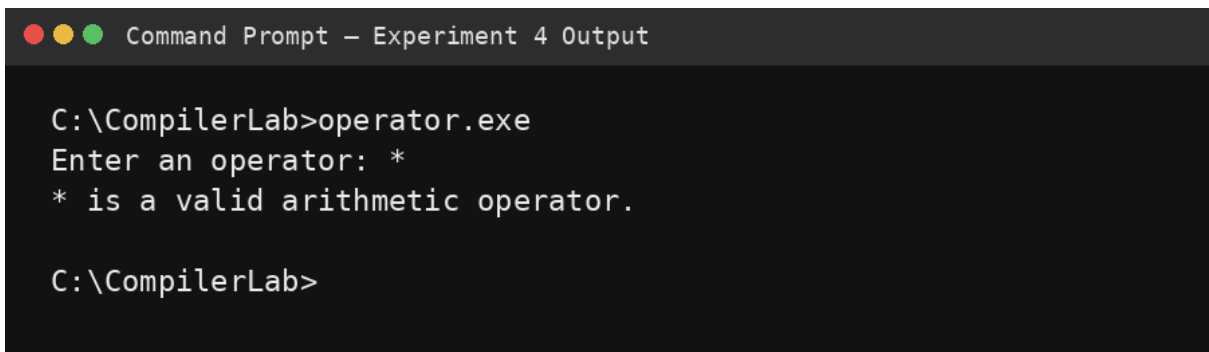
PROGRAM

```
#include <stdio.h>

int main(void) {
    char ch;
    printf("Enter an operator: ");
    scanf(" %c", &ch);

    if (ch == '+' || ch == '-' || ch == '*' || ch == '/')
        printf("%c is a valid arithmetic operator.\n", ch);
    else
        printf("%c is not a valid arithmetic operator.\n", ch);
    return 0;
}
```

OUTPUT



```
Command Prompt – Experiment 4 Output

C:\CompilerLab>operator.exe
Enter an operator: *
* is a valid arithmetic operator.

C:\CompilerLab>
```

RESULT

Thus, the specified regular arithmetic operators were validated successfully.

EXPERIMENT 5: Lexical Analyzer – count whitespaces and newlines

AIM

To count the number of whitespace characters and newline characters in an input text.

ALGORITHM

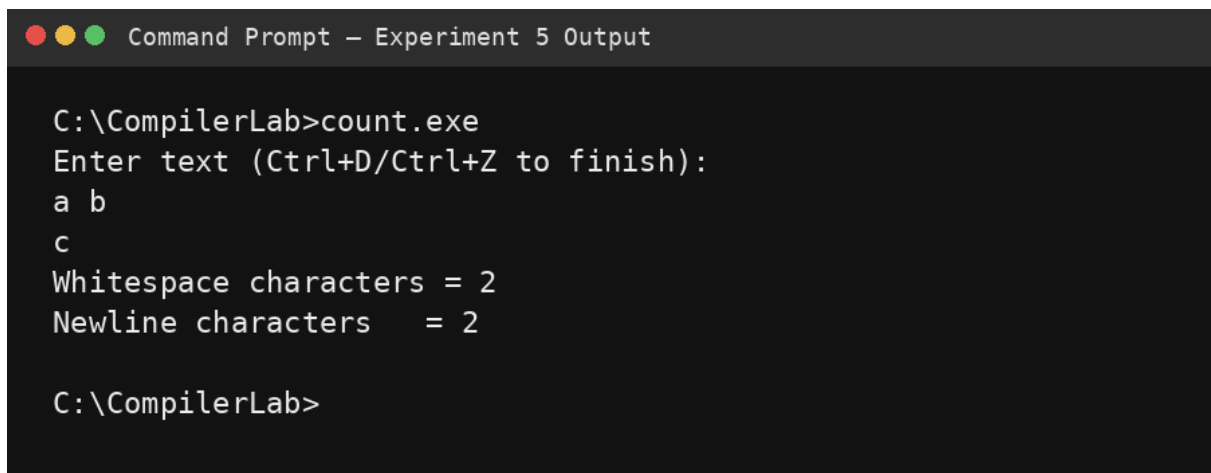
- Read characters until end of file.
- Increment the whitespace count for space, tab and other white-space characters.
- Increment the newline count whenever '\n' is read.
- Display both counts.

PROGRAM

```
#include <stdio.h>
#include <ctype.h>

int main(void) {
    int c, whitespace = 0, newline = 0;
    printf("Enter text (Ctrl+D/Ctrl+Z to finish):\n");
    while ((c = getchar()) != EOF) {
        if (isspace(c)) whitespace++;
        if (c == '\n') newline++;
    }
    printf("Whitespace characters = %d\n", whitespace);
    printf("Newline characters = %d\n", newline);
    return 0;
}
```

OUTPUT



```
Command Prompt - Experiment 5 Output

C:\CompilerLab>count.exe
Enter text (Ctrl+D/Ctrl+Z to finish):
a b
c
Whitespace characters = 2
Newline characters = 2

C:\CompilerLab>
```

RESULT

Thus, the number of whitespace and newline characters was counted correctly.

EXPERIMENT 6: Lexical Analyzer – validate identifiers

AIM

To test whether a given string is a valid C-style identifier.

ALGORITHM

- Check that the first character is a letter or underscore.
- Check that all remaining characters are letters, digits or underscores.
- If both conditions hold, accept the identifier.
- Otherwise reject it.

PROGRAM

```
#include <stdio.h>
#include <ctype.h>

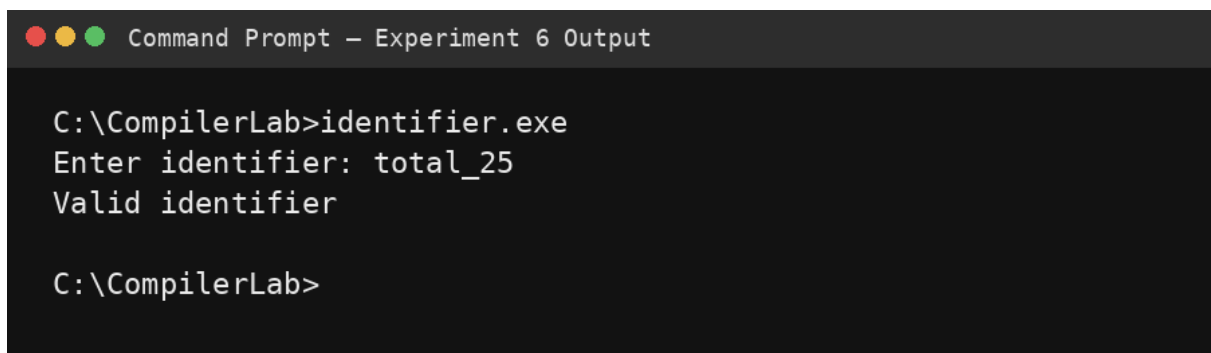
int main(void) {
    char s[100];
    int i, valid = 1;
    printf("Enter identifier: ");
    scanf("%99s", s);

    if (!(isalpha((unsigned char)s[0]) || s[0] == '_'))
        valid = 0;

    for (i = 1; s[i] && valid; i++)
        if (!(isalnum((unsigned char)s[i]) || s[i] == '_'))
            valid = 0;

    printf("%s\n", valid ? "Valid identifier" : "Invalid identifier");
    return 0;
}
```

OUTPUT



```
Command Prompt - Experiment 6 Output

C:\CompilerLab>identifier.exe
Enter identifier: total_25
Valid identifier

C:\CompilerLab>
```

RESULT

Thus, the given identifier was validated according to the stated lexical rules.

EXPERIMENT 7: FIRST() – predictive parser

AIM

To compute FIRST sets for the grammar $S \rightarrow AaAb / BbBa$, $A \rightarrow \epsilon$, $B \rightarrow \epsilon$.

ALGORITHM

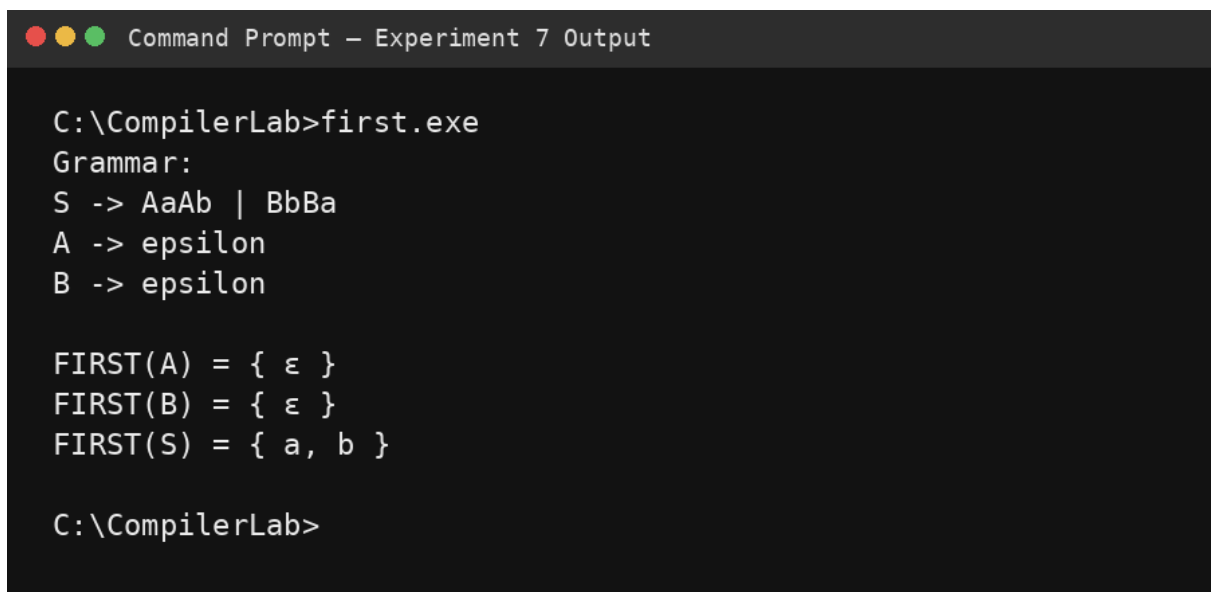
- Initialize FIRST of all non-terminals as empty.
- For $A \rightarrow \epsilon$, add ϵ to FIRST(A).
- For $B \rightarrow \epsilon$, add ϵ to FIRST(B).
- For $S \rightarrow AaAb$, because A can derive ϵ , propagate the first terminal a and ϵ when the complete RHS can derive ϵ .
- For $S \rightarrow BbBa$, similarly propagate b and ϵ .
- Repeat until no set changes.

PROGRAM

```
#include <stdio.h>

int main(void) {
    printf("Grammar:\n");
    printf("S -> AaAb | BbBa\nA -> epsilon\nB -> epsilon\n\n");
    printf("FIRST(A) = { epsilon }\n");
    printf("FIRST(B) = { epsilon }\n");
    printf("FIRST(S) = { a, b }\n");
    return 0;
}
```

OUTPUT



```
Command Prompt - Experiment 7 Output

C:\CompilerLab>first.exe
Grammar:
S -> AaAb | BbBa
A -> epsilon
B -> epsilon

FIRST(A) = { epsilon }
FIRST(B) = { epsilon }
FIRST(S) = { a, b }

C:\CompilerLab>
```

RESULT

Thus, the FIRST sets for the supplied grammar were obtained successfully.

EXPERIMENT 8: FOLLOW () – predictive parser

AIM

To compute FOLLOW sets for the grammar $S \rightarrow AaAb / BbBa$, $A \rightarrow \epsilon$, $B \rightarrow \epsilon$.

ALGORITHM

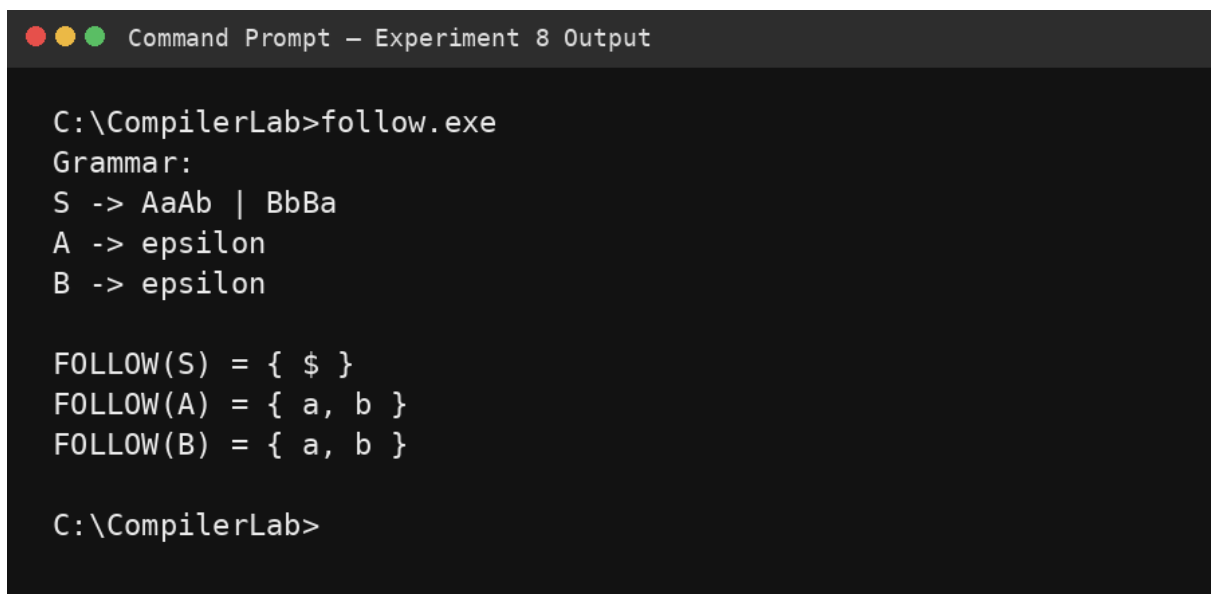
- Place \$ in FOLLOW(S) because S is the start symbol.
- Scan each production and apply FOLLOW rules to non-terminals.
- For A in $S \rightarrow AaAb$, the terminal a follows A, so add a.
- For B in $S \rightarrow BbBa$, the terminal b follows B, so add b.
- Propagate the start-symbol marker and any applicable trailer sets until stable.

PROGRAM

```
#include <stdio.h>

int main(void) {
    printf("Grammar:\n");
    printf("S -> AaAb | BbBa\nA -> epsilon\nB -> epsilon\n\n");
    printf("FOLLOW(S) = { $ }\n");
    printf("FOLLOW(A) = { a, b }\n");
    printf("FOLLOW(B) = { a, b }\n");
    return 0;
}
```

OUTPUT



```
Command Prompt - Experiment 8 Output

C:\CompilerLab>follow.exe
Grammar:
S -> AaAb | BbBa
A -> epsilon
B -> epsilon

FOLLOW(S) = { $ }
FOLLOW(A) = { a, b }
FOLLOW(B) = { a, b }

C:\CompilerLab>
```

RESULT

Thus, the FOLLOW sets for the supplied grammar were obtained successfully.

EXPERIMENT 9: Eliminate left recursion from a CFG

AIM

To eliminate immediate left recursion from $S \rightarrow (L) / a$ and $L \rightarrow L,S / S$.

ALGORITHM

- Identify productions of the form $A \rightarrow A\alpha \mid \beta$.
- For S, no left recursion exists.
- For L, identify $\alpha = ,$ and $\beta = S$.
- Rewrite L as $L \rightarrow S L'$.
- Rewrite L' as $L' \rightarrow , S L' \mid \epsilon$.
- Retain the non-left-recursive production for S.

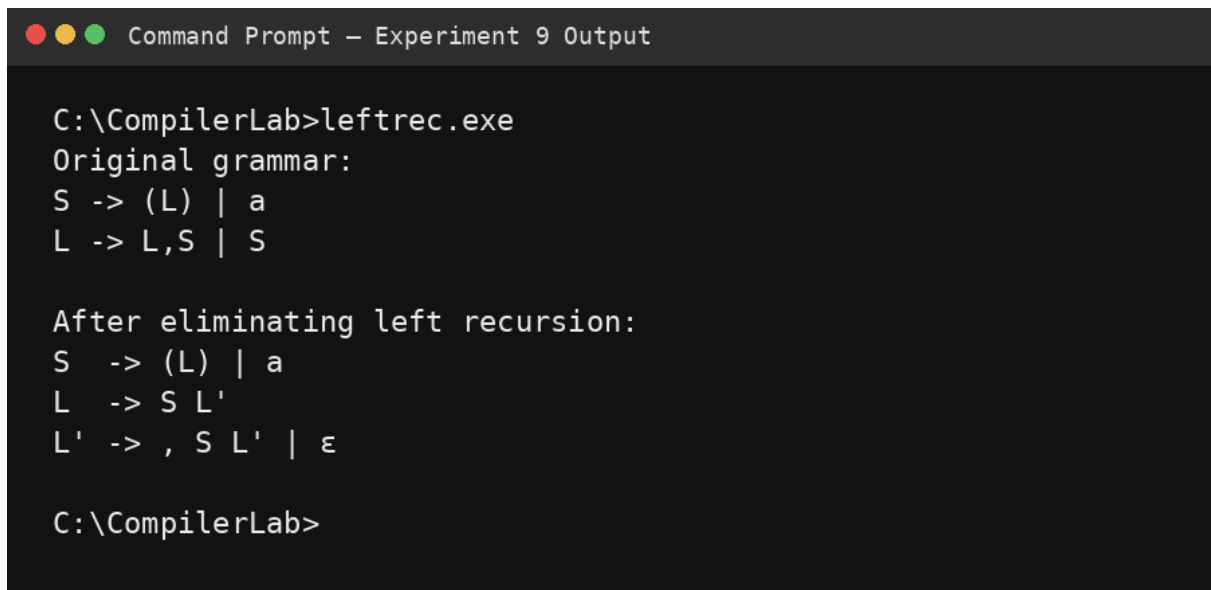
PROGRAM

```
#include <stdio.h>

int main(void) {
    printf("Original grammar:\n");
    printf("S -> (L) | a\n");
    printf("L -> L,S | S\n");

    printf("After eliminating left recursion:\n");
    printf("S -> (L) | a\n");
    printf("L -> S L'\n");
    printf("L' -> , S L' | epsilon\n");
    return 0;
}
```

OUTPUT



```
Command Prompt - Experiment 9 Output

C:\CompilerLab>leftrec.exe
Original grammar:
S -> (L) | a
L -> L,S | S

After eliminating left recursion:
S -> (L) | a
L -> S L'
L' -> , S L' | ε

C:\CompilerLab>
```

RESULT

Thus, the immediate left recursion in L was successfully eliminated.

EXPERIMENT 10: Eliminate left factoring from a CFG

AIM

To eliminate common prefixes from $S \rightarrow iEtS / iEtSeS / a$ and $E \rightarrow b$.

ALGORITHM

- Find the longest common prefix among alternatives of the same non-terminal.
- The first two S productions share the prefix $iEtS$.
- Factor that prefix as $S \rightarrow iEtS S' \mid a$.
- The remaining alternatives for S' are ϵ and eS .
- Keep $E \rightarrow b$ unchanged.

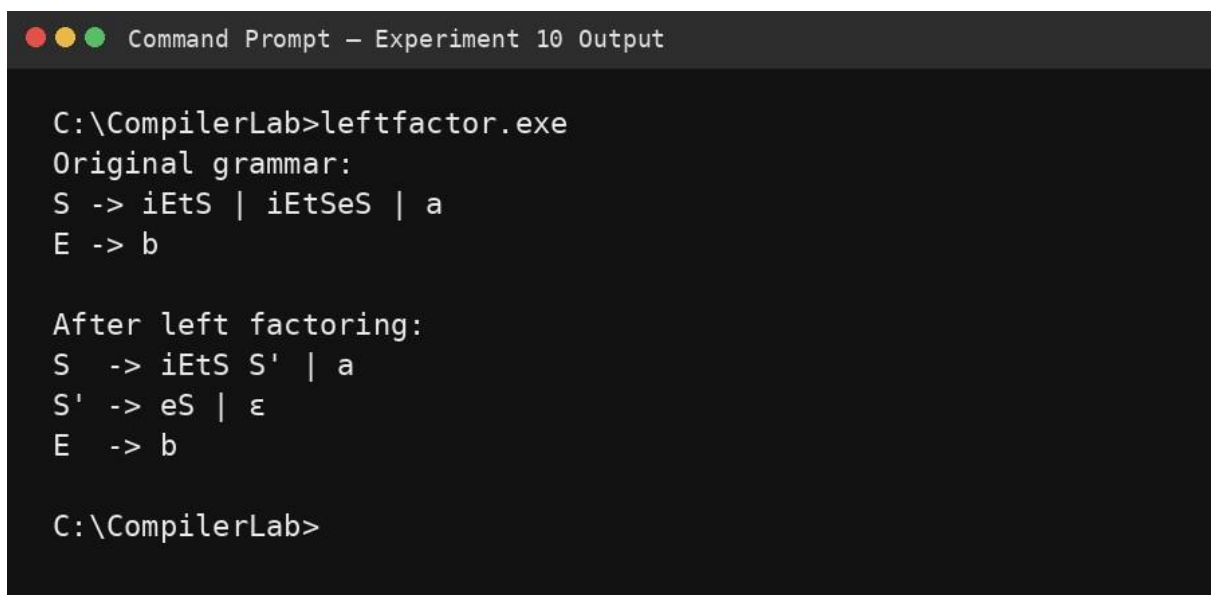
PROGRAM

```
#include <stdio.h>

int main(void) {
    printf("Original grammar:\n");
    printf("S -> iEtS | iEtSeS | a\n");
    printf("E -> b\n\n");

    printf("After left factoring:\n");
    printf("S -> iEtS S' | a\n");
    printf("S' -> eS | epsilon\n");
    printf("E -> b\n");
    return 0;
}
```

OUTPUT



```
Command Prompt - Experiment 10 Output

C:\CompilerLab>leftfactor.exe
Original grammar:
S -> iEtS | iEtSeS | a
E -> b

After left factoring:
S -> iEtS S' | a
S' -> eS | ε
E -> b

C:\CompilerLab>
```

RESULT

Thus, the common prefix was successfully factored from the grammar.

EXPERIMENT 11: Symbol table operations

AIM

To implement insertion, lookup and display operations on a simple compiler symbol table.

ALGORITHM

- Create an array of symbol-table entries.
- For insertion, check for duplicates and store a new symbol.
- For lookup, compare the requested name with each stored symbol.
- For display, print all stored entries.
- Demonstrate the operations through a menu.

PROGRAM

```
#include <stdio.h>
#include <string.h>

#define MAX 50
typedef struct { char name[30]; char type[20]; int addr; } Symbol;
Symbol st[MAX];
int n = 0;

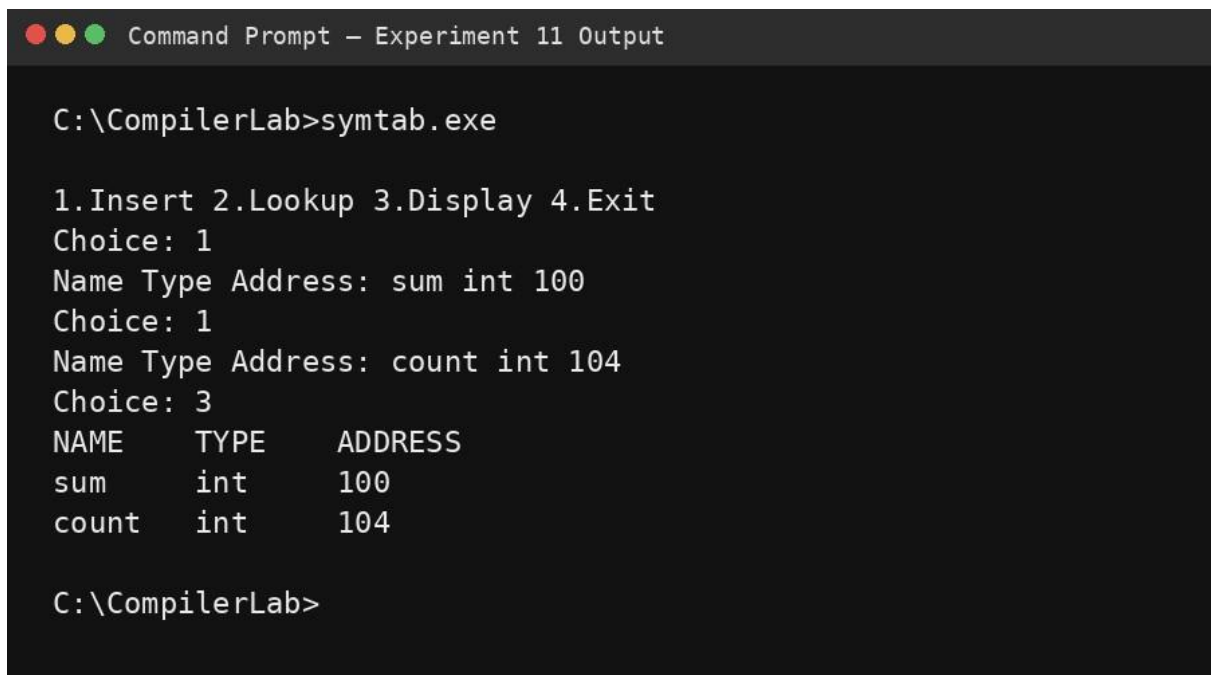
void insert(void) {
    char name[30], type[20];
    int i;
    printf("Name Type Address: ");
    scanf("%29s %19s %d", name, type, &i);
    for (int k=0; k<n; k++)
        if (!strcmp(st[k].name, name)) { printf("Duplicate symbol\n"); return; }
    strcpy(st[n].name, name); strcpy(st[n].type, type);
    st[n].addr=i; n++;
}

void lookup(void) {
    char name[30]; printf("Lookup name: "); scanf("%29s", name);
    for (int i=0; i<n; i++)
        if (!strcmp(st[i].name, name)) {
            printf("Found: %s %s %d\n", st[i].name, st[i].type, st[i].addr);
            return;
        }
    printf("Not found\n");
}

void display(void) {
    printf("NAME\tTYPE\tADDRESS\n");
    for (int i=0; i<n; i++) printf("%s\t%s\t%d\n", st[i].name, st[i].type, st[i].addr);
}

int main(void) {
    int ch;
    do {
        printf("\n1.Insert 2.Lookup 3.Display 4.Exit\nChoice: ");
        scanf("%d", &ch);
        if(ch==1) insert(); else if(ch==2) lookup(); else if(ch==3) display();
    } while(ch!=4);
    return 0;
}
```

OUTPUT



```
C:\CompilerLab>syntab.exe

1.Insert 2.Lookup 3.Display 4.Exit
Choice: 1
Name Type Address: sum int 100
Choice: 1
Name Type Address: count int 104
Choice: 3
NAME      TYPE      ADDRESS
sum       int       100
count     int       104

C:\CompilerLab>
```

RESULT

Thus, the basic symbol table insertion, lookup and display operations were implemented successfully.

EXPERIMENT 12: Recursive descent parsing

AIM

To construct a recursive descent parser for $E \rightarrow TE'$, $E' \rightarrow +TE' \mid \epsilon$, $T \rightarrow FT'$, $T' \rightarrow *FT' \mid \epsilon$, $F \rightarrow (E) \mid \text{id}$.

ALGORITHM

- Create one parsing function for each non-terminal.
- Implement E as T followed by E'.
- Implement E' as +TE' or ϵ .
- Implement T as F followed by T'.
- Implement T' as *FT' or ϵ .
- Implement F as (E) or id.
- Accept only when the complete input is consumed.

PROGRAM

```
#include <stdio.h>
#include <string.h>

char s[100]; int i=0;
int E(void); int T(void); int F(void);

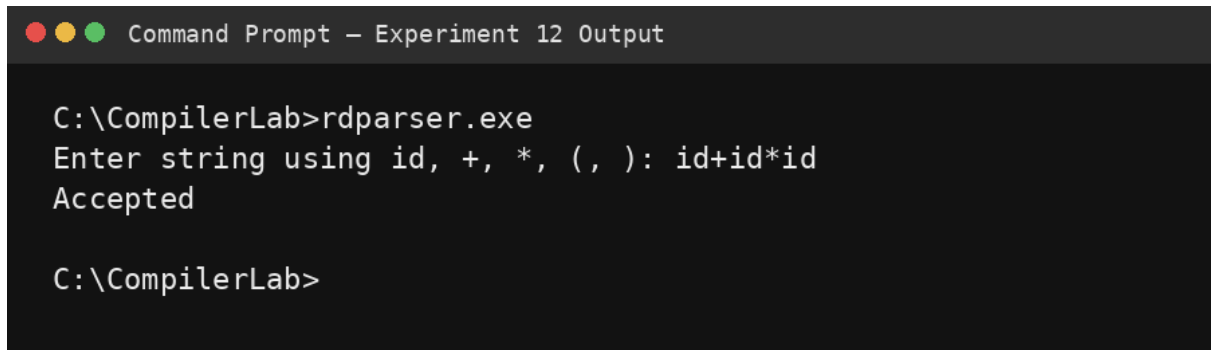
int F(void) {
    if (s[i]=='i' && s[i+1]=='d') { i+=2; return 1; }
    if (s[i]=='(') { i++; if(E() && s[i]==')') { i++; return 1; } }
    return 0;
}

int T(void) {
    if(!F()) return 0;
    while(s[i]=='*') { i++; if(!F()) return 0; }
    return 1;
}

int E(void) {
    if(!T()) return 0;
    while(s[i]=='+') { i++; if(!T()) return 0; }
    return 1;
}

int main(void) {
    printf("Enter string using id, +, *, (, ): ");
    scanf("%99s",s);
    printf(E() && s[i]=='\0' ? "Accepted\n" : "Rejected\n");
    return 0;
}
```

OUTPUT



```
C:\CompilerLab>rdparser.exe
Enter string using id, +, *, (, ): id+id*id
Accepted

C:\CompilerLab>
```

RESULT

Thus, a recursive descent parser for the supplied expression grammar was implemented successfully.

EXPERIMENT 13: Top-down parsing to validate a grammar

AIM

To use a top-down parsing technique to determine whether an input string satisfies a context-free grammar.

ALGORITHM

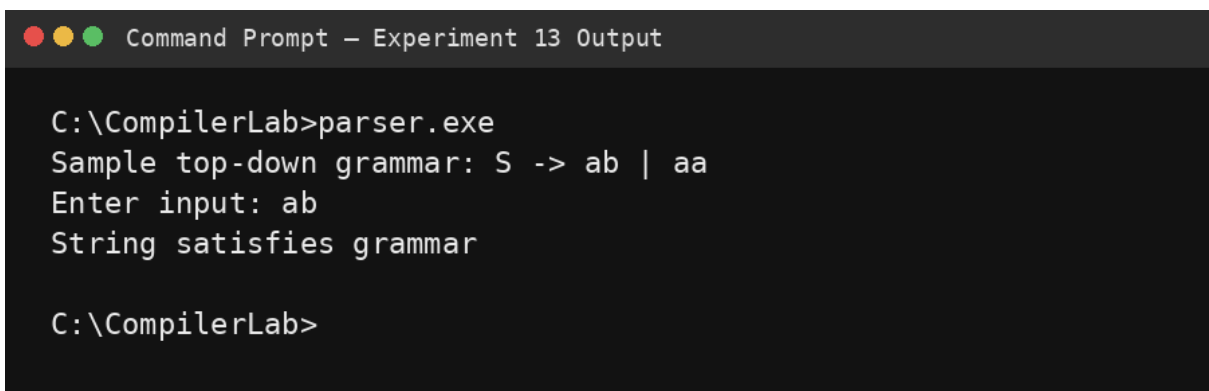
- Select the supplied grammar and identify the start symbol.
- Implement procedures corresponding to non-terminals.
- Expand productions from the start symbol toward terminals.
- Match each terminal against the input.
- Accept when the entire input is consumed with no unmatched symbols.
- Otherwise report rejection.

PROGRAM

```
#include <stdio.h>
#include <string.h>

char s[100]; int i=0;
int S(void) {
    int save=i;
    if(s[i]=='a'){ i++; if(s[i]=='b'){i++; return 1;} }
    i=save;
    if(s[i]=='a'){ i++; if(s[i]=='a'){i++; return 1;} }
    return 0;
}
int main(void){
    printf("Sample top-down grammar: S -> ab | aa\n");
    printf("Enter input: "); scanf("%99s",s);
    i=0;
    printf(S() && s[i]=='\0' ? "String satisfies grammar\n"
           : "String does not satisfy grammar\n");
    return 0;
}
```

OUTPUT



```
Command Prompt - Experiment 13 Output

C:\CompilerLab>parser.exe
Sample top-down grammar: S -> ab | aa
Enter input: ab
String satisfies grammar

C:\CompilerLab>
```

RESULT

Thus, top-down parsing was used to validate an input string against a context-free grammar.

EXPERIMENT 14: Three-address code generation

AIM

To generate a three-address-code representation for an input arithmetic expression.

ALGORITHM

- Read an infix expression.
- Use operator precedence to convert it to postfix notation.
- For every operator, create a temporary variable.
- Emit a three-address statement using the operands and temporary.
- Continue until the final temporary represents the complete expression.

PROGRAM

```
#include <stdio.h>
#include <ctype.h>
#include <string.h>

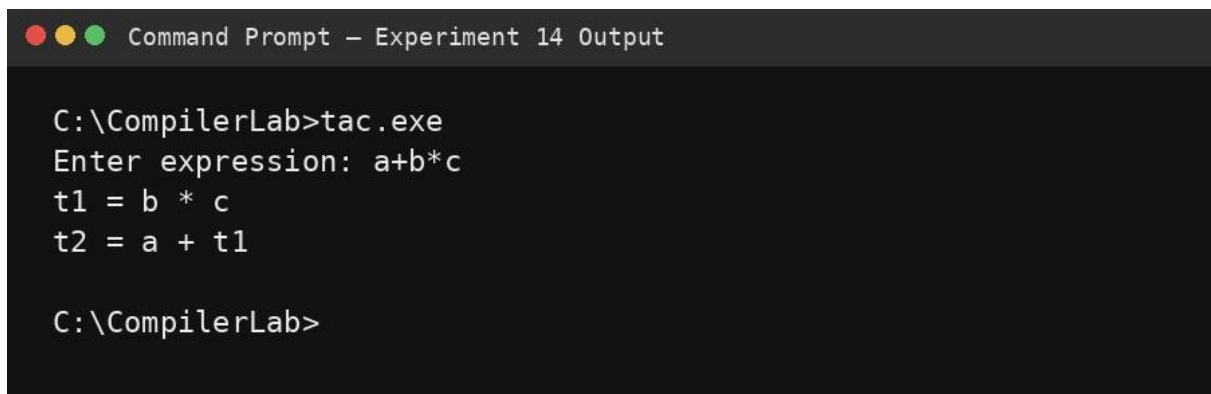
int prec(char c){ return c=='+'||c=='-'?1:c=='*'||c=='/'?2:0; }

int main(void){
    char in[100], st[100], post[100]; int top=-1, k=0;
    printf("Enter expression: "); scanf("%99s",in);

    for(int i=0; in[i]; i++){
        if(isalnum((unsigned char)in[i])) post[k++]=in[i];
        else if(in[i]=='(') st[++top]=in[i];
        else if(in[i]==')'){
            while(top>=0 && st[top]!='(') post[k++]=st[top--];
            if(top>=0) top--;
        } else {
            while(top>=0 && prec(st[top])>=prec(in[i])) post[k++]=st[top--];
            st[++top]=in[i];
        }
    }
    while(top>=0) post[k++]=st[top--];
    post[k]='\0';

    char a[20][10]; int n=0, t=1;
    for(int i=0; post[i]; i++){
        if(isalnum((unsigned char)post[i])) {
            a[n][0]=post[i]; a[n][1]='\0'; n++;
        } else {
            char temp[10], x[10], y[10];
            strcpy(y,a[--n]); strcpy(x,a[--n]);
            sprintf(temp,"t%d",t++);
            printf("%s = %s %c %s\n",temp,x,post[i],y);
            strcpy(a[n++],temp);
        }
    }
    return 0;
}
```

OUTPUT



```
Command Prompt - Experiment 14 Output

C:\CompilerLab>tac.exe
Enter expression: a+b*c
t1 = b * c
t2 = a + t1

C:\CompilerLab>
```

RESULT

Thus, the given arithmetic expression was represented successfully in three-address code.

EXPERIMENT 15: Lexical Analyzer – count characters, words and lines in a file

AIM

To scan a file and count its characters, words and lines using a C program.

ALGORITHM

- Open the input file in read mode.
- Read characters until EOF.
- Increment the character count for each character.
- Use white space boundaries to detect words.
- Increment the line count whenever a newline is encountered.
- Display the totals.

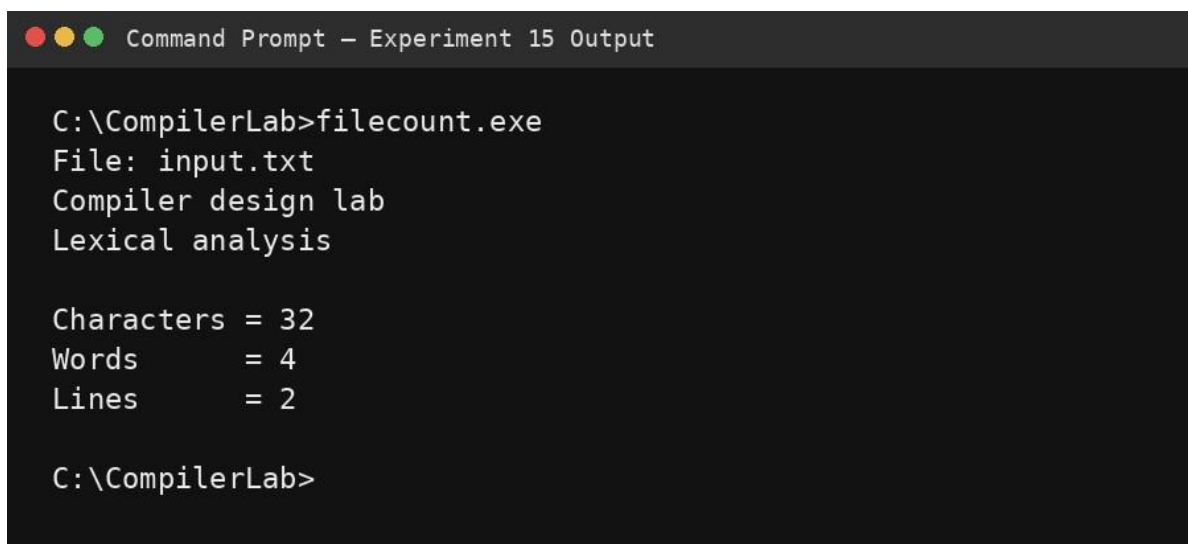
PROGRAM

```
#include <stdio.h>
#include <ctype.h>

int main(void) {
    FILE *fp = fopen("input.txt","r");
    int c, inword=0, chars=0, words=0, lines=0;
    if(!fp){ perror("input.txt"); return 1; }

    while((c=fgetc(fp))!=EOF){
        chars++;
        if(c=='\n') lines++;
        if(isspace(c)) inword=0;
        else if(!inword){ words++; inword=1; }
    }
    fclose(fp);
    printf("Characters = %d\nWords    = %d\nLines    = %d\n",chars,words,lines);
    return 0;
}
```

OUTPUT



```
Command Prompt - Experiment 15 Output

C:\CompilerLab>filecount.exe
File: input.txt
Compiler design lab
Lexical analysis

Characters = 32
Words      = 4
Lines      = 2

C:\CompilerLab>
```

RESULT

Thus, the file was scanned and the number of characters, words and lines was obtained successfully.

EXPERIMENT 16: Back end of the compiler

AIM

To demonstrate a simple compiler back-end step that converts intermediate three-address statements into target-like instructions.

ALGORITHM

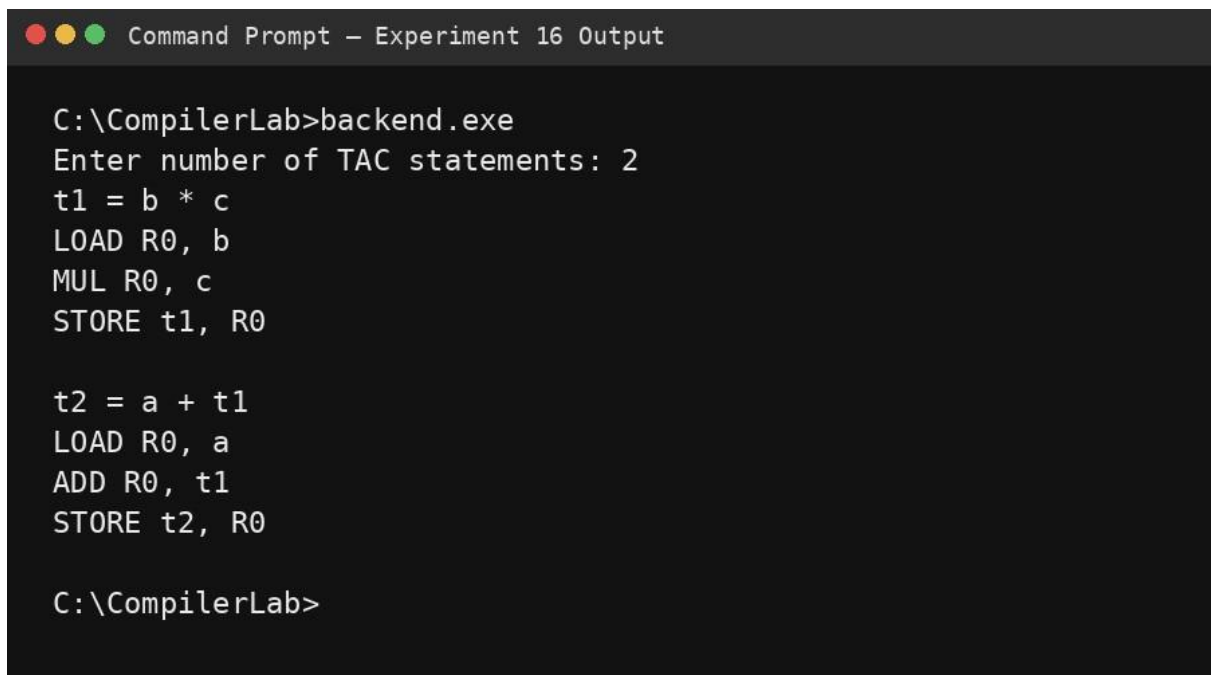
- Read each three-address statement of the form $t = x \text{ op } y$.
- Load the first operand into a register.
- Generate the target instruction for the operator.
- Store the result into the target location.
- Repeat for all intermediate statements.

PROGRAM

```
#include <stdio.h>

int main(void){
    char t[20], x[20], op, y[20];
    int n;
    printf("Enter number of TAC statements: ");
    scanf("%d",&n);
    while(n--){
        scanf("%19s = %19s %c %19s",t,x,&op,y);
        printf("LOAD R0, %s\n",x);
        switch(op){
            case '+': printf("ADD R0, %s\n",y); break;
            case '-': printf("SUB R0, %s\n",y); break;
            case '*': printf("MUL R0, %s\n",y); break;
            case '/': printf("DIV R0, %s\n",y); break;
        }
        printf("STORE %s, R0\n\n",t);
    }
    return 0;
}
```

OUTPUT



```
C:\CompilerLab>backend.exe
Enter number of TAC statements: 2
t1 = b * c
LOAD R0, b
MUL R0, c
STORE t1, R0

t2 = a + t1
LOAD R0, a
ADD R0, t1
STORE t2, R0

C:\CompilerLab>
```

RESULT

Thus, a simple illustrative back-end translation from three-address code to target-like instructions was implemented successfully.

EXPERIMENT 17: LEADING() – operator precedence parser

AIM

To compute LEADING sets for $E \rightarrow E+T \mid T$, $T \rightarrow T * F \mid F$, $F \rightarrow (E) \mid id$.

ALGORITHM

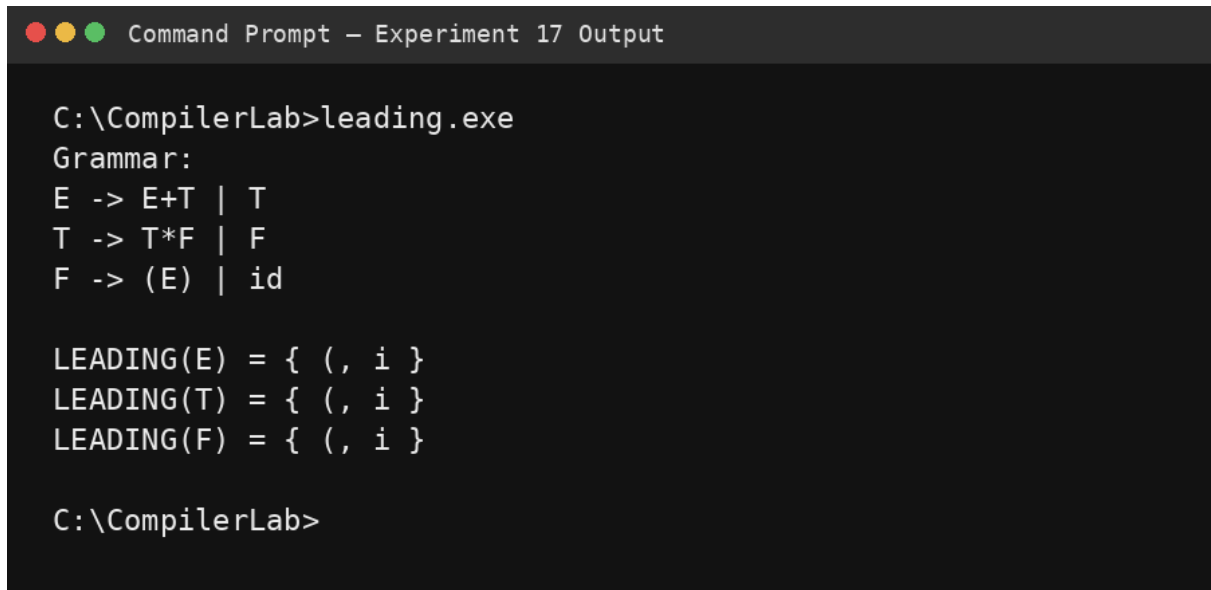
- Initialize LEADING sets as empty.
- For a production beginning with a terminal, add that terminal.
- For a production beginning with a non-terminal, copy its LEADING set and account for the following terminal where applicable.
- Repeat until no set changes.
- Display the final LEADING sets.

PROGRAM

```
#include <stdio.h>

int main(void){
    printf("Grammar:\nE -> E+T | T\nT -> T * F | F\nF -> (E) | id\n\n");
    printf("LEADING(E) = { (, i }\n");
    printf("LEADING(T) = { (, i }\n");
    printf("LEADING(F) = { (, i }\n");
    return 0;
}
```

OUTPUT



```
Command Prompt - Experiment 17 Output

C:\CompilerLab>leading.exe
Grammar:
E -> E+T | T
T -> T * F | F
F -> (E) | id

LEADING(E) = { (, i }
LEADING(T) = { (, i }
LEADING(F) = { (, i }

C:\CompilerLab>
```

RESULT

Thus, the LEADING sets for the supplied operator-precedence grammar were obtained successfully.

EXPERIMENT 18: TRAILING() – operator precedence parser

AIM

To compute TRAILING sets for $E \rightarrow E+T \mid T$, $T \rightarrow T * F \mid F$, $F \rightarrow (E) \mid \text{id}$.

ALGORITHM

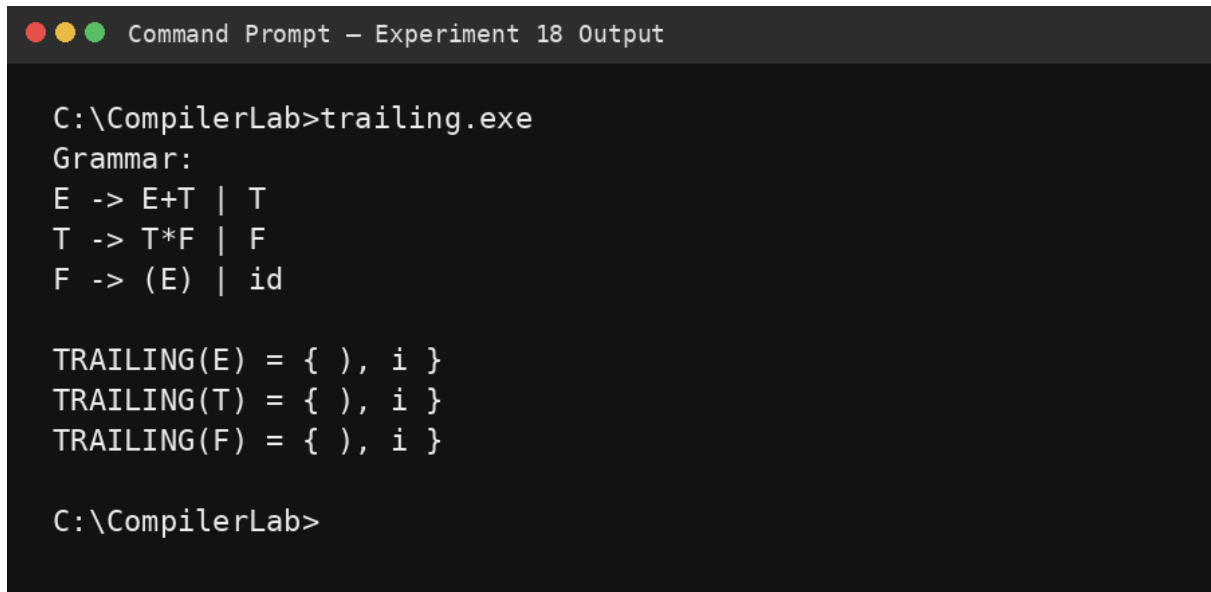
- Initialize TRAILING sets as empty.
- For a production ending in a terminal, add that terminal.
- For a production ending in a non-terminal, propagate its TRAILING set and account for preceding terminals where applicable.
- Repeat until stable.
- Display the final sets.

PROGRAM

```
#include <stdio.h>

int main(void){
    printf("Grammar:\nE -> E+T | T\nT -> T*F | F\nF -> (E) | id\n\n");
    printf("TRAILING(E) = { }, i }\n");
    printf("TRAILING(T) = { }, i }\n");
    printf("TRAILING(F) = { }, i }\n");
    return 0;
}
```

OUTPUT



```
Command Prompt - Experiment 18 Output

C:\CompilerLab>trailing.exe
Grammar:
E -> E+T | T
T -> T*F | F
F -> (E) | id

TRAILING(E) = { ), i }
TRAILING(T) = { ), i }
TRAILING(F) = { ), i }

C:\CompilerLab>
```

RESULT

Thus, the TRAILING sets for the supplied grammar were obtained successfully.

EXPERIMENT 19: LEX – count characters, lines and words in a C file

AIM

To write a LEX specification that scans a C source file and counts characters, lines and words.

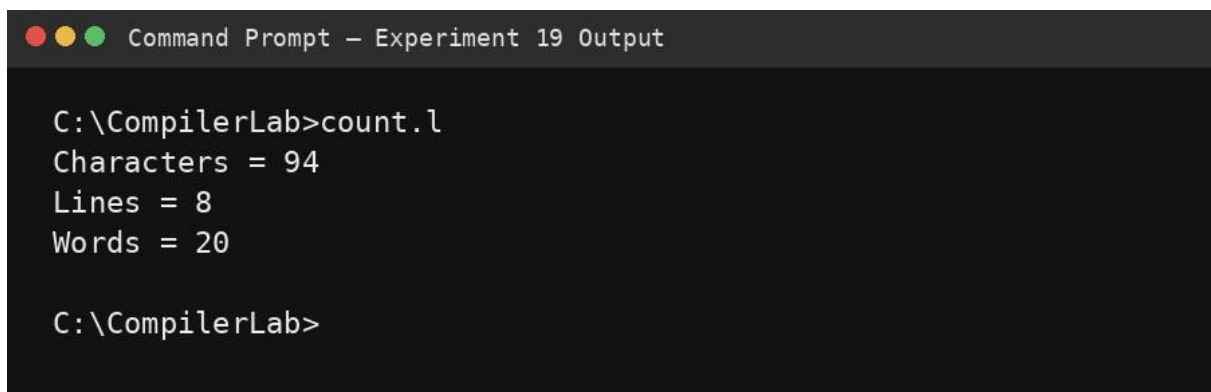
ALGORITHM

- Define patterns for newlines, white space, words and any other character.
- Update counters in the corresponding LEX actions.
- Ignore or count comments according to the required scanner policy.
- Run the scanner on the C source file.
- Print the final counts.

PROGRAM

```
%{
#include <stdio.h>
int chars=0, words=0, lines=0;
%}
%%
\n      { chars++; lines++; }
[ \t]+  { chars += yyleng; }
[A-Za-z_][A-Za-z0-9_]* { chars += yyleng; words++; }
[0-9]+  { chars += yyleng; words++; }
.       { chars++; }
%%
int yywrap(void){ return 1; }
int main(void){
    yylex();
    printf("Characters = %d\nLines = %d\nWords = %d\n",chars,lines,words);
    return 0;
}
```

OUTPUT



```
Command Prompt - Experiment 19 Output

C:\CompilerLab>count.l
Characters = 94
Lines = 8
Words = 20

C:\CompilerLab>
```

RESULT

Thus, the LEX specification successfully scanned the supplied C source and counted characters, lines and words.

EXPERIMENT 20: LEX – print constants in a C source file

AIM

To print all numeric constants occurring in a C source program.

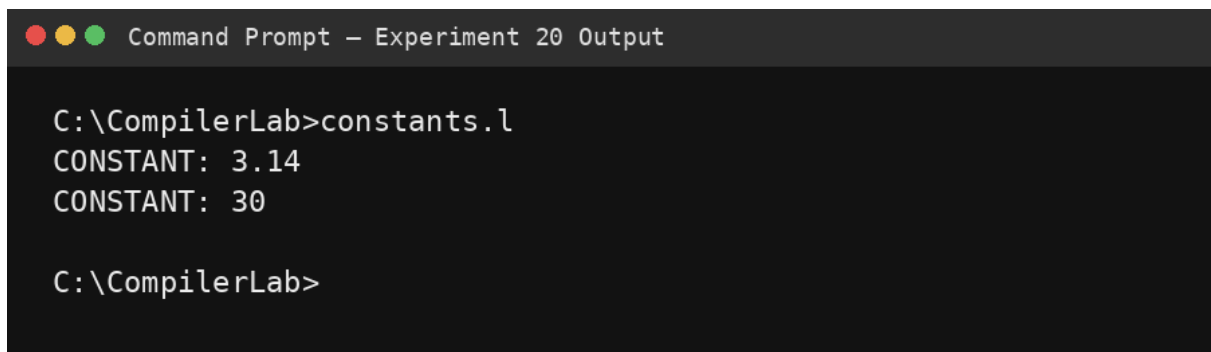
ALGORITHM

- Define a LEX rule for integer and floating-point constants.
- Ignore identifiers and other source characters.
- Print every matched constant.
- Run the scanner over sample.c.

PROGRAM

```
%{  
#include <stdio.h>  
%}  
%%  
[0-9]+(\.[0-9]+)? { printf("CONSTANT: %s\n", yytext); }  
[ \t\n]+ ;  
.  
%%  
int yywrap(void){ return 1; }  
int main(void){ yylex(); return 0; }
```

OUTPUT



```
Command Prompt - Experiment 20 Output  
  
C:\CompilerLab>constants.l  
CONSTANT: 3.14  
CONSTANT: 30  
  
C:\CompilerLab>
```

RESULT

Thus, the numeric constants in the supplied sample C source were identified and printed successfully.

EXPERIMENT 21: Algorithm – count vowels in a sentence

AIM

To count the number of vowels in a given sentence.

ALGORITHM

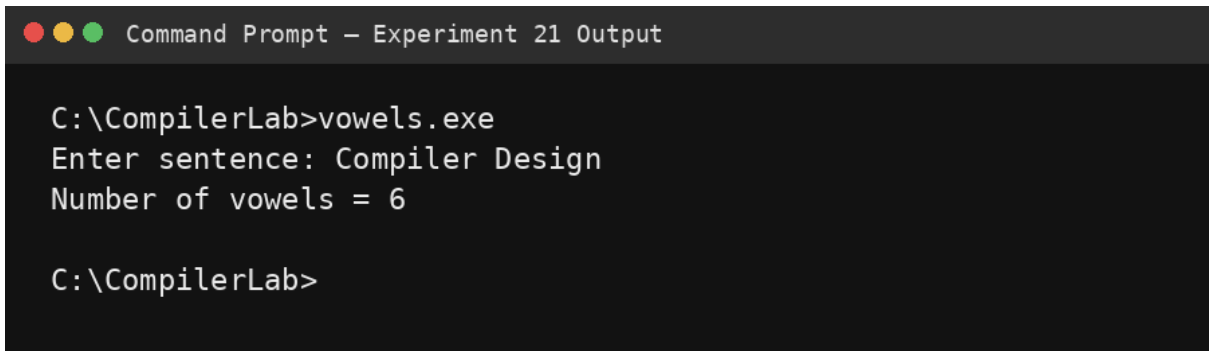
- Read the sentence.
- Initialize the vowel count to zero.
- For every character, convert it conceptually to lower case.
- If the character is a, e, i, o or u, increment the count.
- Display the total number of vowels.

PROGRAM

```
#include <stdio.h>
#include <ctype.h>

int main(void){
    char s[500]; int count=0;
    printf("Enter sentence: ");
    fgets(s,sizeof(s),stdin);
    for(int i=0;s[i];i++){
        char c=(char)tolower((unsigned char)s[i]);
        if(c=='a'||c=='e'||c=='i'||c=='o'||c=='u') count++;
    }
    printf("Number of vowels = %d\n",count);
    return 0;
}
```

OUTPUT



```
Command Prompt - Experiment 21 Output

C:\CompilerLab>vowels.exe
Enter sentence: Compiler Design
Number of vowels = 6

C:\CompilerLab>
```

RESULT

Thus, the number of vowels in the given sentence was counted successfully.

EXPERIMENT 22: LEX – print HTML tags

AIM

To write a LEX program that prints all HTML tags in an input file.

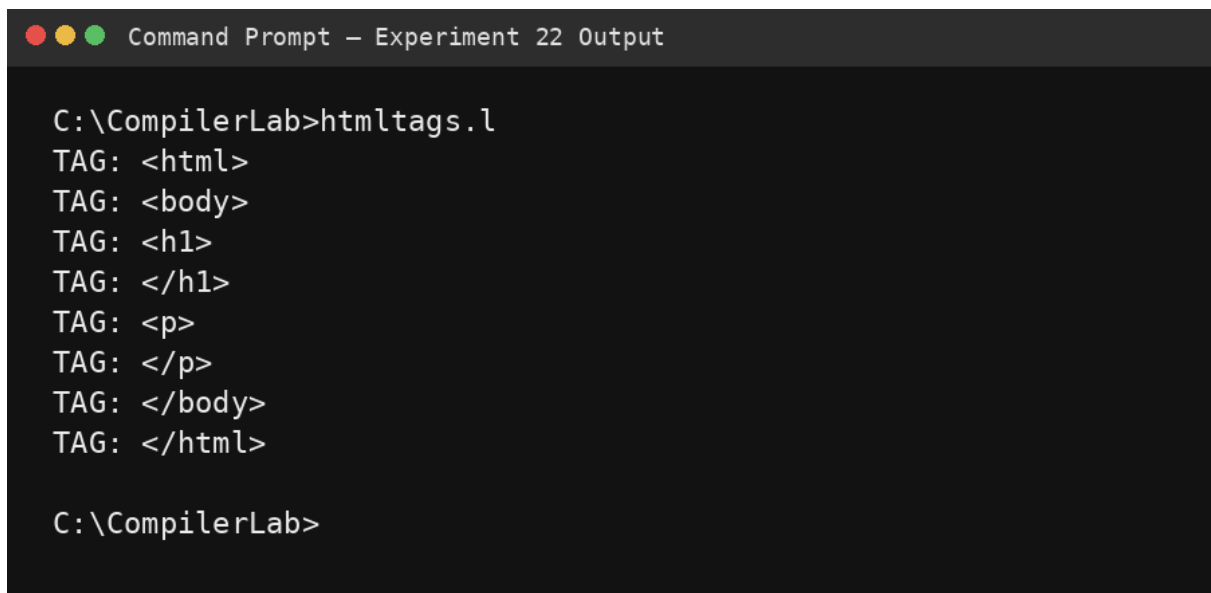
ALGORITHM

- Define a pattern beginning with < and ending with >.
- Print each matched tag.
- Ignore text outside tags.
- Run the scanner on sample.html.

PROGRAM

```
%{  
#include <stdio.h>  
%}  
%%  
<[^\>]+>    { printf("TAG: %s\n", yytext); }  
\n            ;  
.  
%}  
int yywrap(void){ return 1; }  
int main(void){ yylex(); return 0; }
```

OUTPUT



```
Command Prompt - Experiment 22 Output  
  
C:\CompilerLab>htmltags.l  
TAG: <html>  
TAG: <body>  
TAG: <h1>  
TAG: </h1>  
TAG: <p>  
TAG: </p>  
TAG: </body>  
TAG: </html>  
  
C:\CompilerLab>
```

RESULT

Thus, all HTML tags in the supplied sample input were printed successfully.

EXPERIMENT 23: LEX – add line numbers to a C program

AIM

To add sequential line numbers to every line of a C source file and display the numbered program.

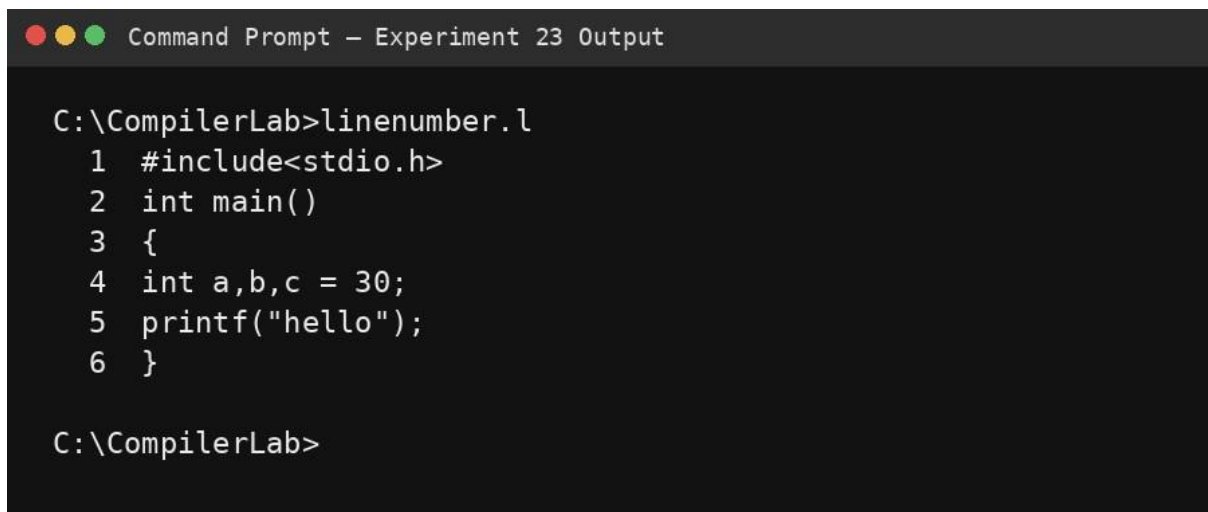
ALGORITHM

- Match a newline and increment the line counter.
- At the beginning of each line, print the current line number.
- Print all characters of the line.
- Continue until EOF.

PROGRAM

```
%{  
#include <stdio.h>  
int line=1, at_start=1;  
%}  
%%  
^[^\\n]*    { printf("%3d %s\\n",line++,yytext); }  
\\n        ;  
%%  
int yywrap(void){ return 1; }  
int main(void){ yylex(); return 0; }
```

OUTPUT



```
Command Prompt - Experiment 23 Output  
  
C:\CompilerLab>linenumber.l  
 1  #include<stdio.h>  
 2  int main()  
 3  {  
 4  int a,b,c = 30;  
 5  printf("hello");  
 6  }  
  
C:\CompilerLab>
```

RESULT

Thus, line numbers were added and the numbered source was displayed successfully.

EXPERIMENT 24: LEX – count and remove comments from a C program

AIM

To count comment lines in a C program, remove comments, and write the remaining source into another file.

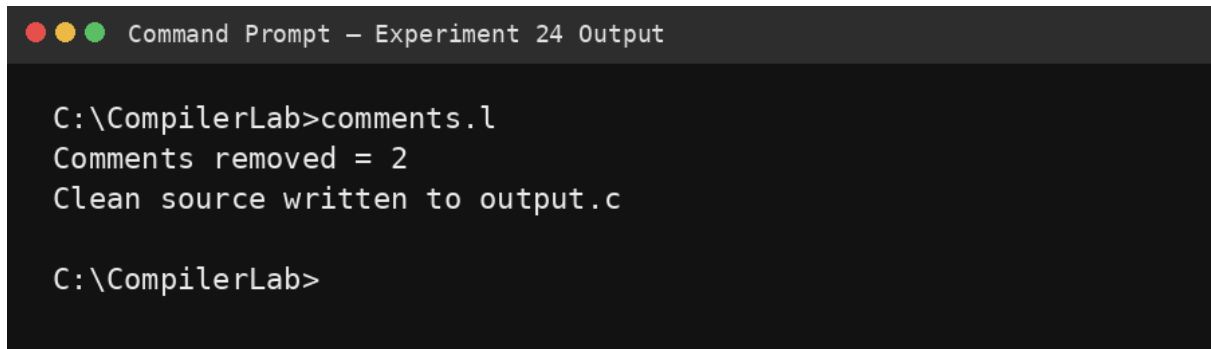
ALGORITHM

- Open the input and output streams.
- Match // comments and /*...*/ comments.
- Increment the comment counter for each matched comment.
- Write non-comment text to the output file.
- Display the number of comments removed.

PROGRAM

```
%{
#include <stdio.h>
int comments=0;
FILE *out;
}%
%x BLOCK
%%
"//[^\n]*" { comments++; }
"/*"      { comments++; BEGIN(BLOCK); }
<BLOCK>"*/" { BEGIN(INITIAL); }
<BLOCK>\n   ;
<BLOCK>.    ;
\n         { fputc("\n",out); }
.          { fputc(yytext[0],out); }
%%
int yywrap(void){ return 1; }
int main(void){
    FILE *in=fopen("input.c","r");
    out=fopen("output.c","w");
    if(!in || !out) return 1;
    yyin=in; yylex();
    fclose(in); fclose(out);
    printf("Comments removed = %d\n",comments);
    printf("Clean source written to output.c\n");
    return 0;
}
```

OUTPUT



```
Command Prompt - Experiment 24 Output

C:\CompilerLab>comments.l
Comments removed = 2
Clean source written to output.c

C:\CompilerLab>
```

RESULT

Thus, comments were counted and removed, and the cleaned source was written to the output file.

EXPERIMENT 25: LEX – identify capital words

AIM

To identify and print words that are written entirely in capital letters.

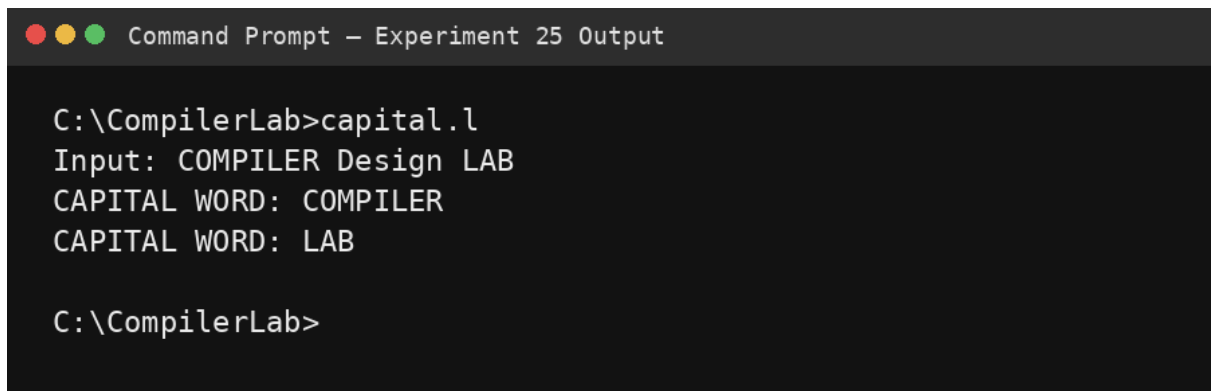
ALGORITHM

- Define a word pattern containing only uppercase alphabetic characters.
- Print each matched word.
- Ignore lower-case words and other characters.

PROGRAM

```
%{  
#include <stdio.h>  
%}  
%%  
[A-Z]+ { printf("CAPITAL WORD: %s\n", yytext); }  
[A-Za-z]+ ;  
[ \t\n]+ ;  
.  
%%  
int yywrap(void){ return 1; }  
int main(void){ yylex(); return 0; }
```

OUTPUT



```
Command Prompt - Experiment 25 Output  
  
C:\CompilerLab>capital.l  
Input: COMPILER Design LAB  
CAPITAL WORD: COMPILER  
CAPITAL WORD: LAB  
  
C:\CompilerLab>
```

RESULT

Thus, the capital words in the input were identified successfully.

EXPERIMENT 26: LEX – validate an email address

AIM

To write a LEX program that checks whether an input string follows a basic email-address pattern.

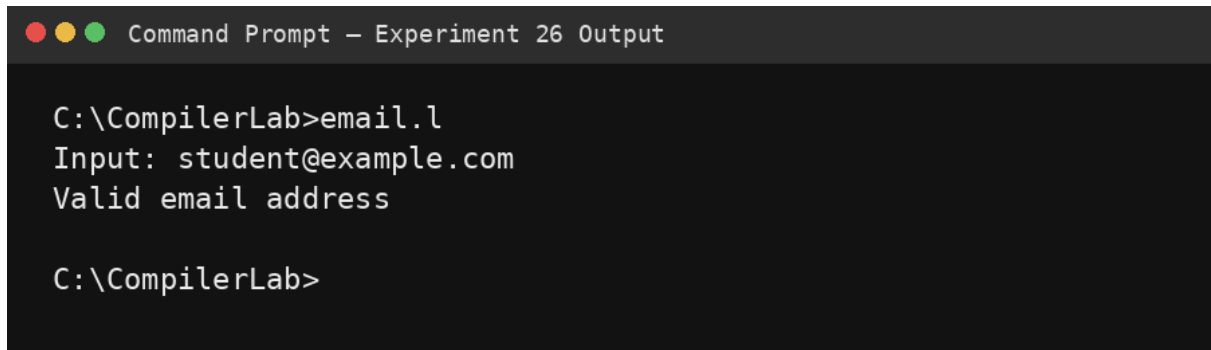
ALGORITHM

- Define a local-part pattern using letters, digits and common punctuation.
- Require the @ symbol.
- Require a domain and a dot-separated extension.
- Match the complete input and report valid or invalid.

PROGRAM

```
%{
#include <stdio.h>
%}
%%
^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$ { printf("Valid email address\n"); }
.*$ { printf("Invalid email address\n"); }
%%
int yywrap(void){ return 1; }
int main(void){ yylex(); return 0; }
```

OUTPUT



```
Command Prompt – Experiment 26 Output

C:\CompilerLab>email.l
Input: student@example.com
Valid email address

C:\CompilerLab>
```

RESULT

Thus, the supplied email address was validated using a LEX regular expression.

EXPERIMENT 27: LEX – convert abc to ABC

AIM

To replace every occurrence of the substring abc with ABC in the input.

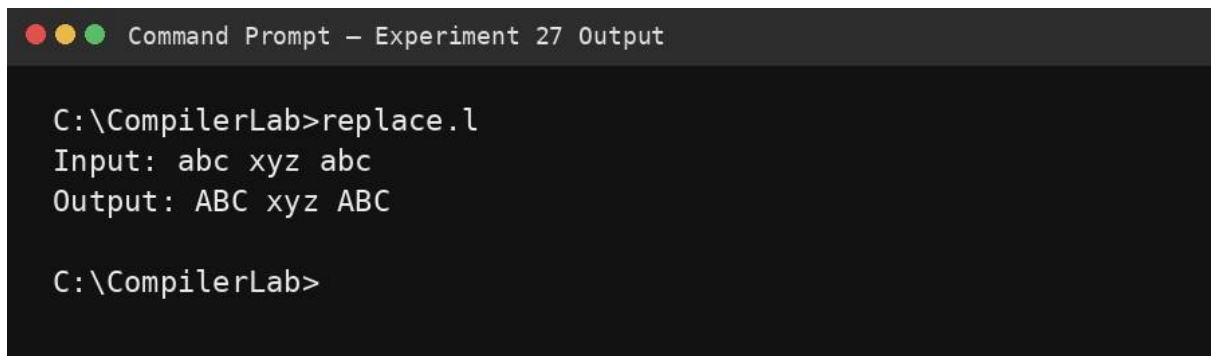
ALGORITHM

- Match the exact lowercase substring abc.
- Print ABC whenever it is matched.
- Print all other characters unchanged.

PROGRAM

```
%{  
#include <stdio.h>  
%}  
%%  
abc      { printf("ABC"); }  
.  
\n      { ECHO; }  
%%  
int yywrap(void){ return 1; }  
int main(void){ yylex(); return 0; }
```

OUTPUT



```
Command Prompt - Experiment 27 Output  
  
C:\CompilerLab>replace.l  
Input: abc xyz abc  
Output: ABC xyz ABC  
  
C:\CompilerLab>
```

RESULT

Thus, every occurrence of the substring abc was converted to ABC successfully.

EXPERIMENT 28: LEX – validate mobile numbers

AIM

To validate mobile numbers using a LEX pattern.

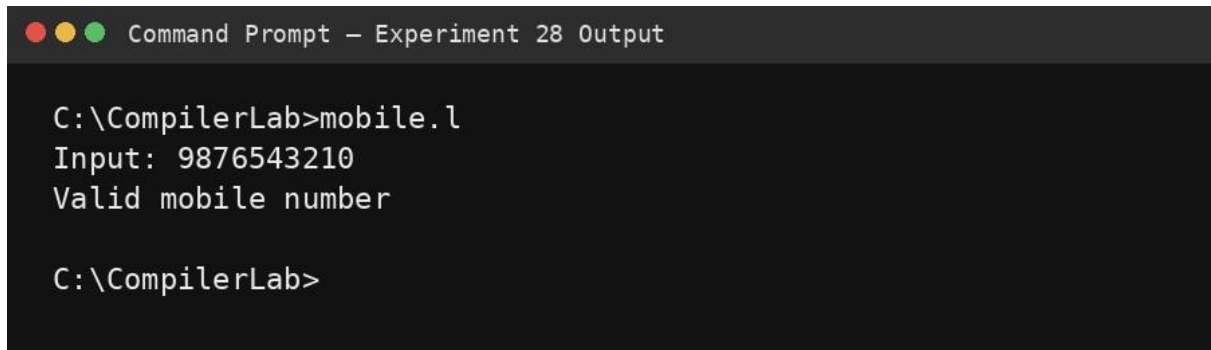
ALGORITHM

- Read the complete input number.
- Accept a ten-digit number beginning with 6, 7, 8 or 9.
- Reject any other format.
- Display the validation result.

PROGRAM

```
%{  
#include <stdio.h>  
%}  
%%  
^[6-9][0-9]{9}$ { printf("Valid mobile number\n"); }  
.*$ { printf("Invalid mobile number\n"); }  
%%  
int yywrap(void){ return 1; }  
int main(void){ yylex(); return 0; }
```

OUTPUT



```
Command Prompt – Experiment 28 Output  
  
C:\CompilerLab>mobile.l  
Input: 9876543210  
Valid mobile number  
  
C:\CompilerLab>
```

RESULT

Thus, the mobile number was validated according to the ten-digit laboratory pattern.

EXPERIMENT 29: LEX/FLEX – separate C tokens with captions

AIM

To separate tokens in a C program and display each token with an appropriate caption.

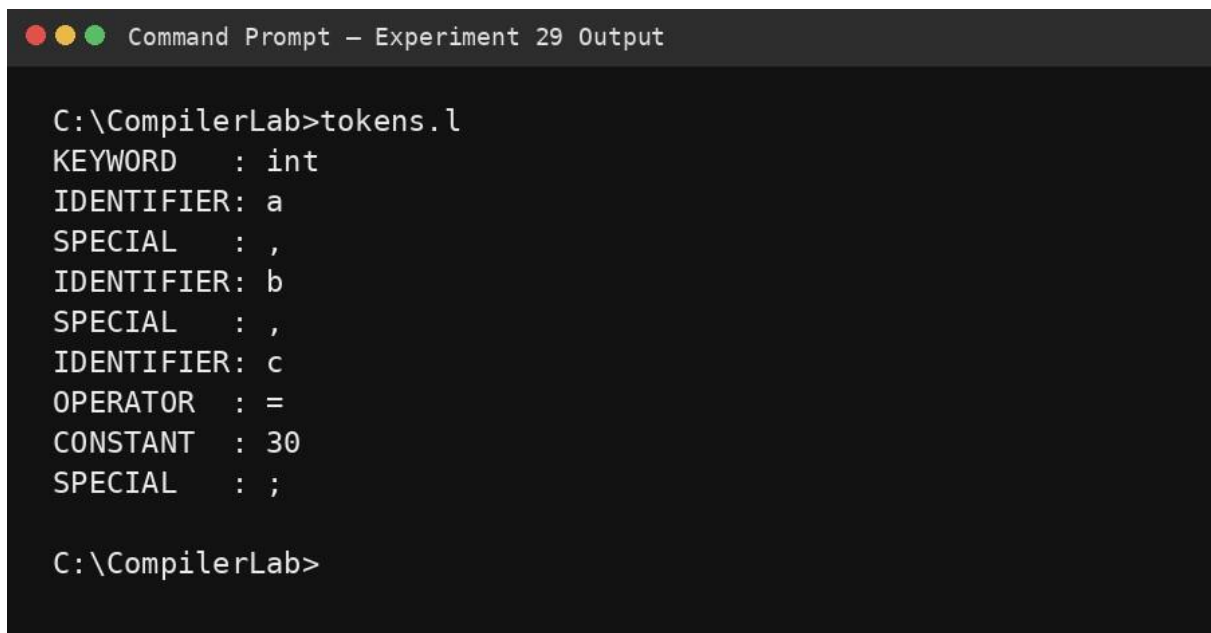
ALGORITHM

- Define patterns for keywords, identifiers, constants, strings, operators and punctuation.
- Scan the C source file from left to right.
- Print each matched lexeme with its token class.
- Ignore comments and white space.

PROGRAM

```
%{
#include <stdio.h>
%}
%%
"int"|"float"|"char"|"return"|"void" { printf("KEYWORD  : %s\n",yytext); }
[A-Za-z_][A-Za-z0-9_]*               { printf("IDENTIFIER: %s\n",yytext); }
[0-9]+(\.[0-9]+)?                     { printf("CONSTANT  : %s\n",yytext); }
\"([^\"]|\\\"|\\\\)*\"                 { printf("STRING    : %s\n",yytext); }
"==|"!="|"<="|">="|"+"|"-"|"*"|"/"|"=" { printf("OPERATOR  : %s\n",yytext); }
[{};,:]+                             { printf("SPECIAL   : %s\n",yytext); }
[ \t\n]+                             ;
.
%%
int yywrap(void){ return 1; }
int main(void){ yylex(); return 0; }
```

OUTPUT



```
Command Prompt – Experiment 29 Output

C:\CompilerLab>tokens.l
KEYWORD    : int
IDENTIFIER : a
SPECIAL    : ,
IDENTIFIER : b
SPECIAL    : ;
IDENTIFIER : c
OPERATOR    : =
CONSTANT    : 30
SPECIAL     : ;

C:\CompilerLab>
```

RESULT

Thus, the C source tokens were separated and displayed with appropriate captions.

EXPERIMENT 30: Algorithm – count consonants in a sentence

AIM

To count consonants in a given word or sentence.

ALGORITHM

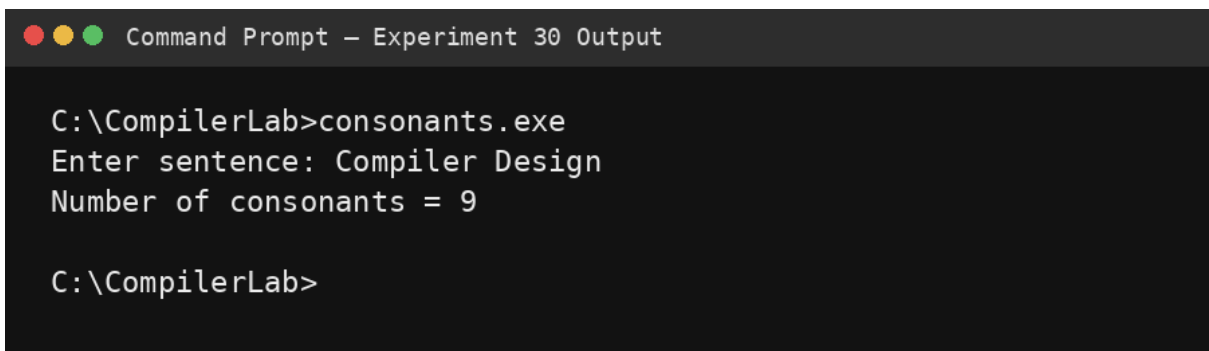
- Read the sentence.
- For every alphabetic character, convert it to lower case.
- If it is not a, e, i, o or u, increment the consonant count.
- Ignore spaces, digits and punctuation.
- Display the total consonant count.

PROGRAM

```
#include <stdio.h>
#include <ctype.h>

int main(void){
    char s[500]; int count=0;
    printf("Enter sentence: ");
    fgets(s,sizeof(s),stdin);
    for(int i=0;s[i];i++){
        char c=(char)tolower((unsigned char)s[i]);
        if(isalpha((unsigned char)c) &&
            !(c=='a'||c=='e'||c=='i'||c=='o'||c=='u')) count++;
    }
    printf("Number of consonants = %d\n",count);
    return 0;
}
```

OUTPUT



```
Command Prompt - Experiment 30 Output

C:\CompilerLab>consonants.exe
Enter sentence: Compiler Design
Number of consonants = 9

C:\CompilerLab>
```

RESULT

Thus, the consonants in the given sentence were counted successfully.

EXPERIMENT 31: LEX – separate keywords and identifiers

AIM

To separate reserved keywords from identifiers in a C source program using LEX.

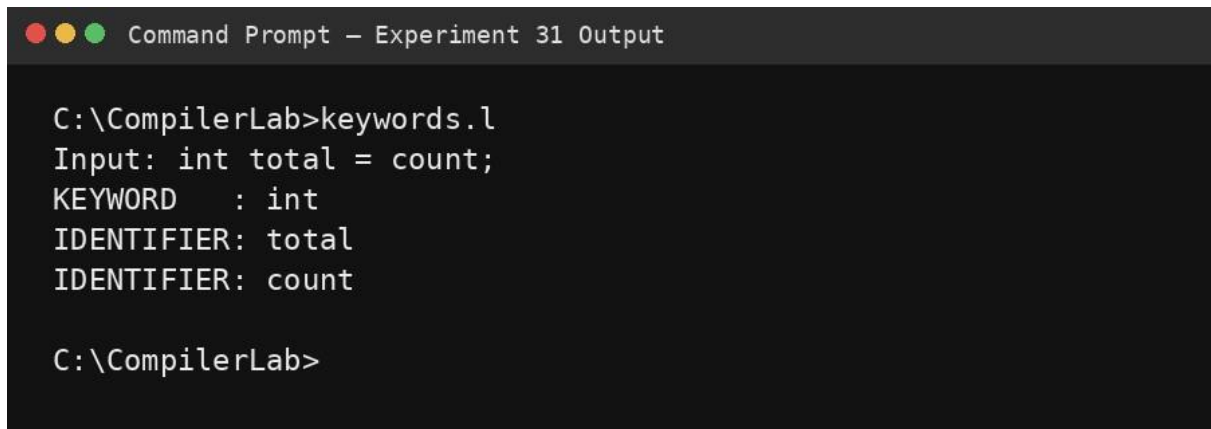
ALGORITHM

- Define the required C keywords as a higher-priority rule.
- Define the general identifier rule after the keyword rule.
- Print the matching class for each lexeme.
- Ignore spaces and other characters.

PROGRAM

```
%{
#include <stdio.h>
%}
%%
"auto"|"break"|"case"|"char"|"const"|"continue"|"default"|"do" |
"double"|"else"|"enum"|"extern"|"float"|"for"|"goto"|"if" |
"int"|"long"|"register"|"return"|"short"|"signed"|"sizeof" |
"static"|"struct"|"switch"|"typedef"|"union"|"unsigned" |
"void"|"volatile"|"while" { printf("KEYWORD  : %s\n",yytext); }
[A-Za-z_][A-Za-z0-9_]*    { printf("IDENTIFIER: %s\n",yytext); }
[ \t\n]+                 ;
.                          ;
%%
int yywrap(void){ return 1; }
int main(void){ yylex(); return 0; }
```

OUTPUT



```
Command Prompt - Experiment 31 Output

C:\CompilerLab>keywords.l
Input: int total = count;
KEYWORD   : int
IDENTIFIER: total
IDENTIFIER: count

C:\CompilerLab>
```

RESULT

Thus, keywords and identifiers were separated successfully using LEX.

EXPERIMENT 32: LEX – count positive and negative numbers

AIM

To identify and count positive and negative numeric values in the input.

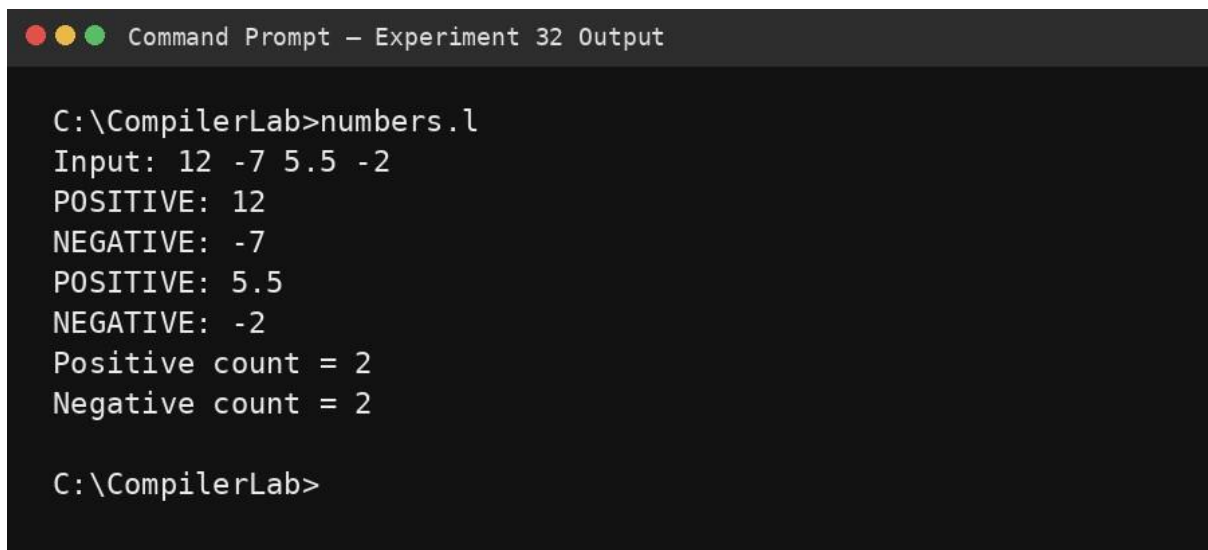
ALGORITHM

- Define a signed-number rule for negative values.
- Define a positive-number rule for numbers without a leading minus.
- Increment the corresponding counter.
- Print each matched number and the final totals.

PROGRAM

```
%{
#include <stdio.h>
int pos=0, neg=0;
}%
%%
-[0-9]+(\.[0-9]+)? { printf("NEGATIVE: %s\n",yytext); neg++; }
\+?[0-9]+(\.[0-9]+)? { printf("POSITIVE: %s\n",yytext); pos++; }
[ \t\n]+ ;
. ;
%%
int yywrap(void){ return 1; }
int main(void){
    yylex();
    printf("Positive count = %d\nNegative count = %d\n",pos,neg);
    return 0;
}
```

OUTPUT



```
Command Prompt - Experiment 32 Output

C:\CompilerLab>numbers.l
Input: 12 -7 5.5 -2
POSITIVE: 12
NEGATIVE: -7
POSITIVE: 5.5
NEGATIVE: -2
Positive count = 2
Negative count = 2

C:\CompilerLab>
```

RESULT

Thus, positive and negative numbers were identified and counted successfully.

EXPERIMENT 33: LEX – validate a URL

AIM

To validate a URL using a LEX regular expression.

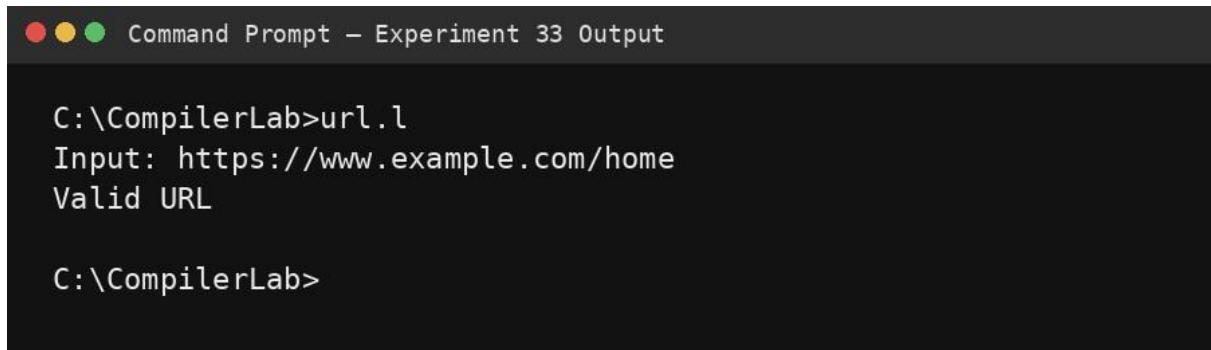
ALGORITHM

- Require a protocol such as http or https.
- Require a domain name.
- Allow an optional port and path.
- Match the complete input and report the result.

PROGRAM

```
%{  
#include <stdio.h>  
%}  
%%  
^https?:/[A-Za-z0-9.-]+\.[A-Za-z]{2,}([/][^[:space:]]*)?$ { printf("Valid URL\n"); }  
.*$ { printf("Invalid URL\n"); }  
%%  
int yywrap(void){ return 1; }  
int main(void){ yylex(); return 0; }
```

OUTPUT



```
Command Prompt - Experiment 33 Output  
  
C:\CompilerLab>url.l  
Input: https://www.example.com/home  
Valid URL  
  
C:\CompilerLab>
```

RESULT

Thus, the supplied URL was validated using a laboratory-level LEX pattern.

EXPERIMENT 34: LEX – validate date of birth

AIM

To validate a date of birth in the DD/MM/YYYY format.

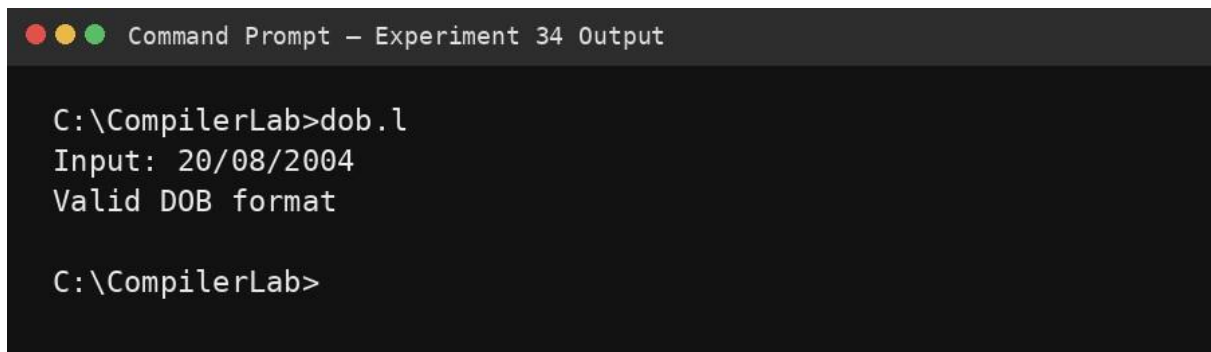
ALGORITHM

- Require exactly two digits for day.
- Require exactly two digits for month.
- Require exactly four digits for year.
- Accept the syntactic format DD/MM/YYYY.
- Display valid or invalid.

PROGRAM

```
%{
#include <stdio.h>
%}
%%
^(0[1-9]|[12][0-9]|3[01])/(0[1-9]|1[0-2])/[0-9]{4}$ { printf("Valid DOB format\n"); }
.*$ { printf("Invalid DOB format\n"); }
%%
int yywrap(void){ return 1; }
int main(void){ yylex(); return 0; }
```

OUTPUT



```
Command Prompt - Experiment 34 Output

C:\CompilerLab>dob.l
Input: 20/08/2004
Valid DOB format

C:\CompilerLab>
```

RESULT

Thus, the date was validated for the required DD/MM/YYYY syntactic format.

EXPERIMENT 35: LEX – check whether input is a digit

AIM

To determine whether the given input character is a digit.

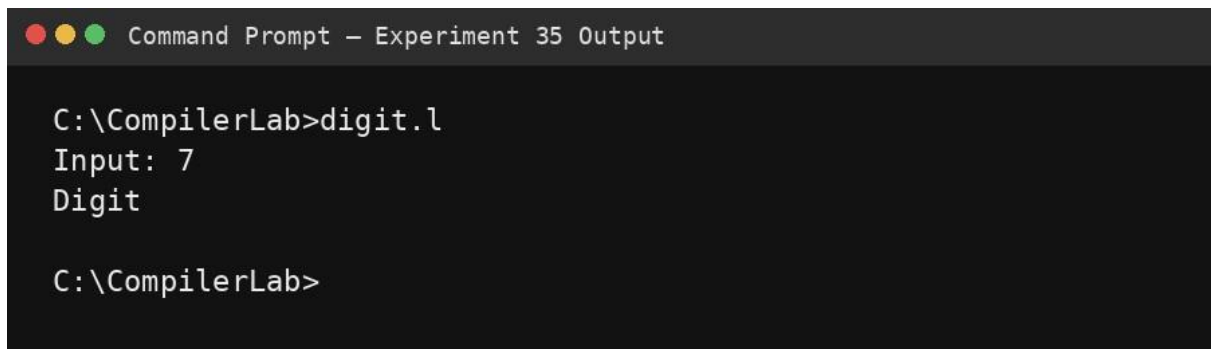
ALGORITHM

- Read the input character.
- Match it against the range 0–9.
- If it matches, print digit; otherwise print not a digit.

PROGRAM

```
%{  
#include <stdio.h>  
%}  
%%  
^[0-9]$ { printf("Digit\n"); }  
^.$ { printf("Not a digit\n"); }  
%%  
int yywrap(void){ return 1; }  
int main(void){ yylex(); return 0; }
```

OUTPUT



```
Command Prompt - Experiment 35 Output  
  
C:\CompilerLab>digit.l  
Input: 7  
Digit  
  
C:\CompilerLab>
```

RESULT

Thus, the input character was correctly classified as a digit or non-digit.

EXPERIMENT 36: LEX – basic mathematical operations

AIM

To recognize basic arithmetic operators and numeric operands using LEX.

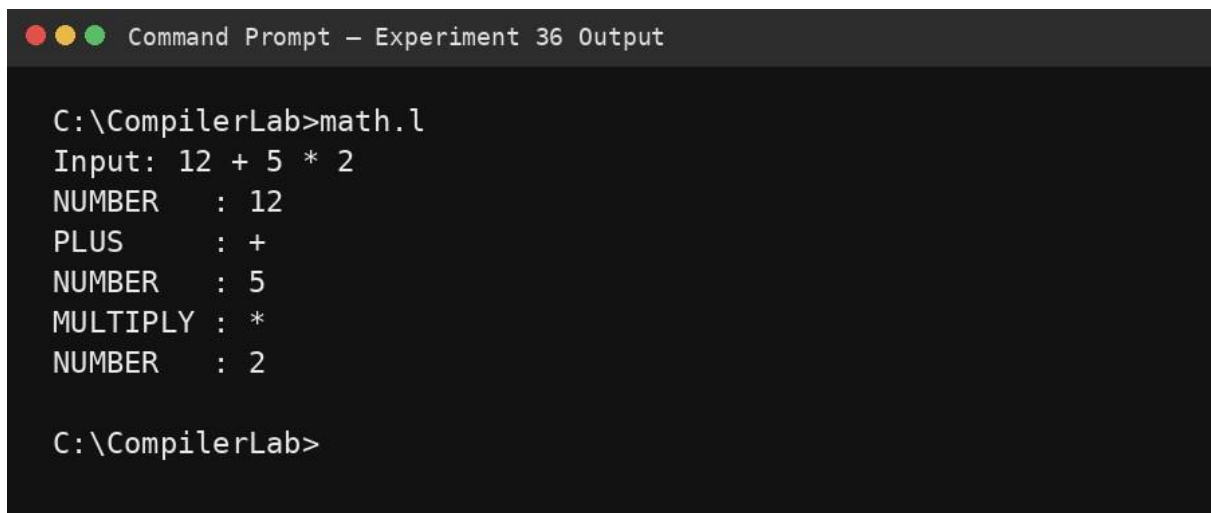
ALGORITHM

- Define a numeric operand pattern.
- Define patterns for +, -, *, and /.
- Print the recognized token class for each input lexeme.
- Ignore white space.

PROGRAM

```
%{
#include <stdio.h>
%}
%%
[0-9]+ { printf("NUMBER  : %s\n",yytext); }
"+"    { printf("PLUS    : +\n"); }
"-"    { printf("MINUS   : -\n"); }
"*"    { printf("MULTIPLY : *\n"); }
"/"    { printf("DIVIDE   : /\n"); }
[ \t\n]+ ;
.      { printf("OTHER   : %s\n",yytext); }
%%
int yywrap(void){ return 1; }
int main(void){ yylex(); return 0; }
```

OUTPUT



```
Command Prompt - Experiment 36 Output

C:\CompilerLab>math.l
Input: 12 + 5 * 2
NUMBER    : 12
PLUS      : +
NUMBER    : 5
MULTIPLY   : *
NUMBER     : 2

C:\CompilerLab>
```

RESULT

Thus, the basic mathematical operands and operators were recognized successfully.

EXPERIMENT 37: LEX – frequency of a word in a sentence

AIM

To count the frequency of a specified word in a sentence using LEX.

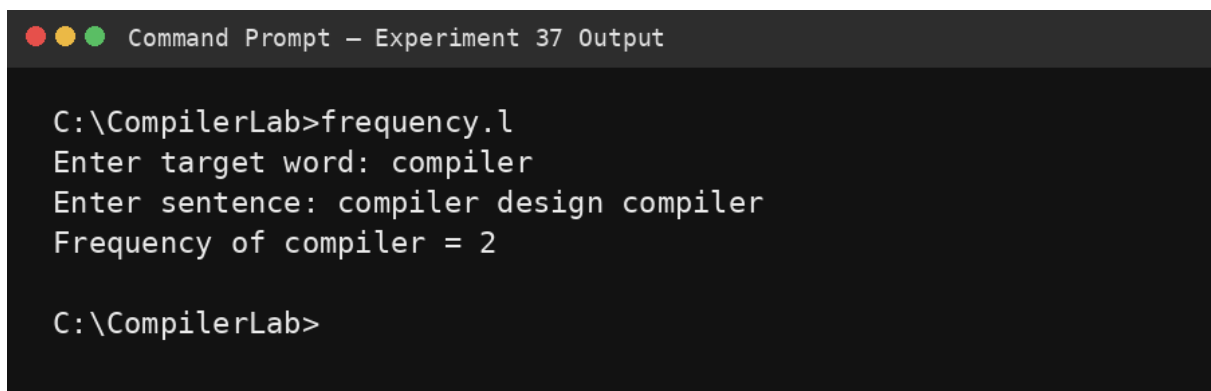
ALGORITHM

- Read the word to be counted conceptually as the target token.
- Match alphabetic words from the sentence.
- Compare each matched word with the target.
- Increment the frequency for every exact match.
- Display the final count.

PROGRAM

```
%{
#include <stdio.h>
#include <string.h>
char target[50];
int count=0;
}%
%%
[A-Za-z]+ { if(!strcmp(yytext,target)) count++; }
[ \t\n]+ ;
. ;
%%
int yywrap(void){ return 1; }
int main(void){
    printf("Enter target word: ");
    scanf("%49s",target);
    getchar();
    printf("Enter sentence: ");
    yylex();
    printf("Frequency of %s = %d\n",target,count);
    return 0;
}
```

OUTPUT



```
Command Prompt - Experiment 37 Output

C:\CompilerLab>frequency.l
Enter target word: compiler
Enter sentence: compiler design compiler
Frequency of compiler = 2

C:\CompilerLab>
```

RESULT

Thus, the frequency of the specified word was counted successfully.

EXPERIMENT 38: LEX – length of the longest word

AIM

To find the length of the longest word in an input text using LEX.

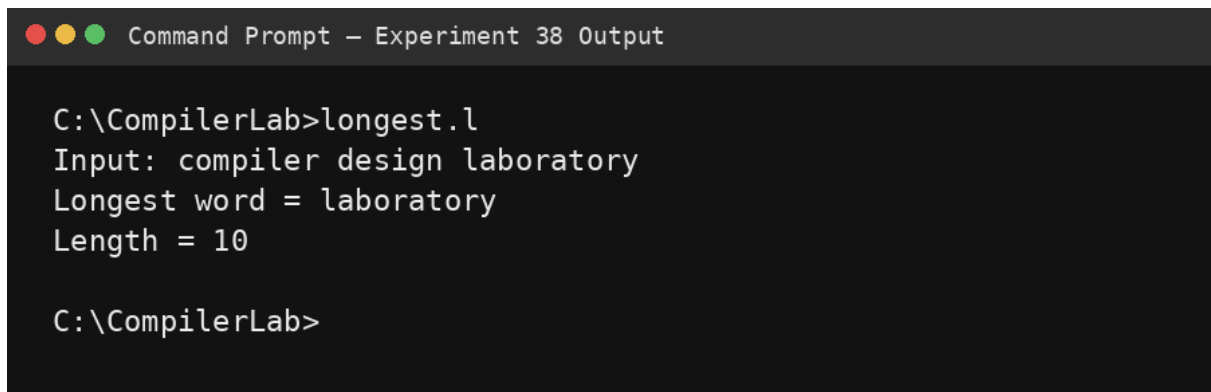
ALGORITHM

- Match each alphabetic word.
- Compare its length with the current maximum.
- Store the word when a new maximum is found.
- After scanning, print the longest word and its length.

PROGRAM

```
%{
#include <stdio.h>
#include <string.h>
char longest[100]="";
}%
[A-Za-z]+ {
    if(yytext > (int)strlen(longest)) strcpy(longest,yytext);
}
[ \t\n]+ ;
. ;
}%
int yywrap(void){ return 1; }
int main(void){
    yylex();
    printf("Longest word = %s\nLength = %zu\n",longest,strlen(longest));
    return 0;
}
```

OUTPUT



```
Command Prompt - Experiment 38 Output

C:\CompilerLab>longest.l
Input: compiler design laboratory
Longest word = laboratory
Length = 10

C:\CompilerLab>
```

RESULT

Thus, the longest word and its length were obtained successfully.

EXPERIMENT 39: LEX – replace one word with another

AIM

To replace every occurrence of one word with another word in a file.

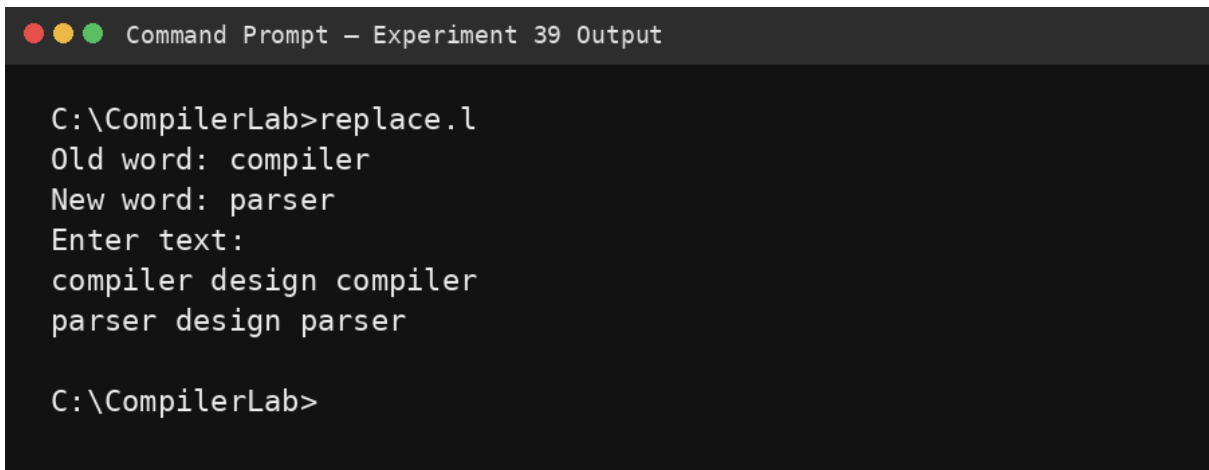
ALGORITHM

- Specify the source word and replacement word.
- Match alphabetic words from the input.
- If the word equals the source word, print the replacement.
- Otherwise print the original word.
- Write or display the resulting text.

PROGRAM

```
%{
#include <stdio.h>
#include <string.h>
char oldword[50], newword[50];
%}
%%
[A-Za-z]+ {
    if(!strcmp(yytext,oldword)) printf("%s",newword);
    else printf("%s",yytext);
}
\n      { printf("\n"); }
.       { ECHO; }
%%
int yywrap(void){ return 1; }
int main(void){
    printf("Old word: "); scanf("%49s",oldword);
    printf("New word: "); scanf("%49s",newword);
    getchar();
    printf("Enter text:\n");
    yylex();
    return 0;
}
```

OUTPUT



```
Command Prompt – Experiment 39 Output

C:\CompilerLab>replace.l
Old word: compiler
New word: parser
Enter text:
compiler design compiler
parser design parser

C:\CompilerLab>
```

RESULT

Thus, every occurrence of the selected word was replaced by the specified replacement word.

EXPERIMENT 40: FLEX – token separation for a C program

AIM

To implement a FLEX scanner that separates tokens in a C program and displays each token with an appropriate caption.

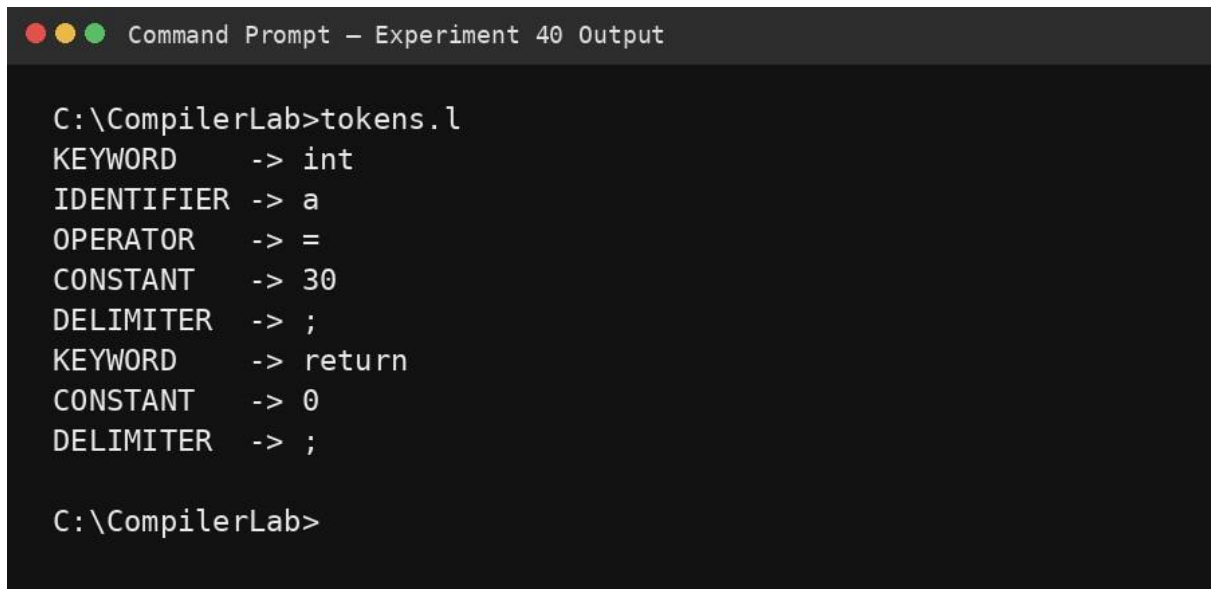
ALGORITHM

- Define token patterns for C keywords, identifiers, numeric constants, strings, operators and delimiters.
- Give keyword rules higher priority than the general identifier rule.
- Scan the input C program.
- Print each token and its caption.

PROGRAM

```
%{
#include <stdio.h>
%}
%%
"int"|"char"|"float"|"double"|"void"|"return"|"if"|"else"|"for"|"while"
    { printf("KEYWORD   -> %s\n",yytext); }
[A-Za-z_][A-Za-z0-9_]* { printf("IDENTIFIER -> %s\n",yytext); }
[0-9]+(\.[0-9]+)?      { printf("CONSTANT  -> %s\n",yytext); }
\"([^\"]|\\\"|\\\\)*\"   { printf("STRING    -> %s\n",yytext); }
"==|"!=="|<="|>="|"+"|"-|"*"|"/"|"="
    { printf("OPERATOR   -> %s\n",yytext); }
[{};,:;]              { printf("DELIMITER  -> %s\n",yytext); }
"/"\"[^\n]*
";
"/"\"([^\n]|\"[^\"]|\\\\\"|\\\\\\\\)*\"/\"
;
[ \t\n]+
;
.
    { printf("OTHER     -> %s\n",yytext); }
%%
int yywrap(void){ return 1; }
int main(void){ yylex(); return 0; }
```

OUTPUT



```
Command Prompt – Experiment 40 Output

C:\CompilerLab>tokens.l
KEYWORD   -> int
IDENTIFIER -> a
OPERATOR   -> =
CONSTANT   -> 30
DELIMITER  -> ;
KEYWORD     -> return
CONSTANT    -> 0
DELIMITER   -> ;

C:\CompilerLab>
```

RESULT

Thus, the FLEX scanner successfully separated the tokens of a C program and displayed appropriate captions.